



LAXMI INSTITUTE OF TECHNOLOGY

Laxmi Vidyapeeth, Sarigam

GUJARAT TECHNOLOGICAL UNIVERSITY

Laxmi Institute of Technology (LIT), Sarigam — Laxmi Vidyapeeth

Department of Computer Science & Engineering

A MINI PROJECT REPORT

ON

"ExpenseTracker"

A Full-Stack MERN Personal Expense & Budget Management System

"Track every rupee."

Submitted by

SHARMA SAURABH

Enrollment No. **250863131007**

Semester VII, Computer Science & Engineering

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF ENGINEERING IN COMPUTER SCIENCE & ENGINEERING

Gujarat Technological University

Academic Year 2025 – 2026

Project Guide	_____ (Guide Name)
Head of Department	_____
Institute	Laxmi Institute of Technology, Sarigam
Report Date	08 October 2026

Abstract

Managing personal money with notebooks or scattered spreadsheets is slow, error-prone and gives no real picture of spending habits. **ExpenseTracker** is a web application built on the MERN stack — **MongoDB, Express.js, React.js and Node.js** — that lets a user record income and expenses, organise them into categories, set monthly budgets, track savings goals, automate recurring payments and understand spending through interactive charts and month-over-month reports.

The system exposes a REST API secured with JWT authentication, uses a Redux-Toolkit-powered React front end and stores data in MongoDB. Beyond basic CRUD it provides smart de-duplicated notifications, monthly reporting with percentage change, CSV export, receipt uploads, tags, favorites, light/dark mode and eight selectable colour themes. All data is strictly per-user and every amount is validated on both the client and the server, so malformed values never reach the database. The application runs entirely on a local machine with two commands and depends on no third-party hosted service.

Keywords: MERN stack, personal finance, expense tracker, budget, savings goal, recurring expense, JWT authentication, Redux Toolkit, MongoDB aggregation, React, Express.

Project Overview

ExpenseTracker is a complete, working personal-finance web application built by a single developer as a Semester-VII mini project. A user signs up, records every income and expense, organises them into categories, sets a monthly limit per category, creates savings goals and recurring bills — and the application automatically reports where the money went through charts, budgets status and month-over-month comparisons.

What the product does

- **Record** — income/expense entries with category, payment method, tags, notes, receipt image and favourite flag.
- **Control** — per-category monthly budgets that turn yellow at $\geq 80\%$ and red at $\geq 100\%$, with warnings that never repeat.
- **Automate** — recurring bills post themselves as real transactions the moment they fall due.
- **Save** — savings goals with contributions, progress bars and a milestone alert at 100%.
- **Understand** — dashboard, five chart types, and a monthly report that says "you spent 48% less than last month".
- **Export** — one-click CSV download of any filtered list or month.

Project at a glance

Metric	Value
Architecture	3-tier (React $\rightleftharpoons$ Express $\rightleftharpoons$ MongoDB)
REST endpoints	41 (38 authenticated)
Database collections	7
Frontend pages / routes	11 pages / 12 routes
Redux slices	8
Colour themes + dark mode	8 + on/off
Reported test cases passed	20 / 20
Screens & code figures in this report	15 + 7

Module snapshot

Group	Modules
Capture	Authentication (JWT) · Transactions · Categories · Receipt upload · Favourites · Tags
Control	Monthly budgets · Budget notifications · Savings goals · Recurring expenses
Insight	Dashboard · Analytics (5 chart types) · Monthly reports · CSV export
Platform	REST API + security layer · Redux state · Theme engine (8 themes, dark mode) · UX polish (skeletons, empty states, micro-interactions)

How to see it in 60 seconds: start the backend (`cd server && npm run seed && npm run dev`), start the frontend (`cd client && npm run dev`), open `http://localhost:5173` and log in as `demo@example.com / password123` — the account is pre-loaded with 32 transactions, budgets, goals and two already-due recurring bills so every screen has content.

Table of Contents

—	Certificate	2
—	Declaration	3
—	Acknowledgement	3
—	Abstract	4
—	Project Overview	5
—	Index of Figures	7
1	Introduction & Problem Definition	9
2	Objectives, Scope & Feasibility	10
3	System Analysis — Requirements & Use Cases	11
4	Technology Stack & Justification	13
5	System Design — Architecture, Data Flow & Structure	14
6	Database Design — ER Diagram & Schema	18
7	Module Description & Feature Study	20
8	API Reference	23
9	User Interface, Design System & Screenshots	26
10	Implementation Highlights (with code)	37
11	Security & Data Validation	48
12	Testing & Test Cases	49
13	Challenges Faced & Solutions	52
14	Limitations & Future Scope	53
15	Conclusion	54
—	References	54
A	Appendix A — How to Run & Demo Credentials	55
B	Appendix B — Environment Variables & Seed Data	55
C	Appendix C — Code Metrics & Module Index	56

Index of Figures

Figure	Caption
1	Use-Case Diagram of ExpenseTracker
2	Three-tier architecture of ExpenseTracker
3	Context diagram (Level-0 DFD) of ExpenseTracker
4	Level-1 data flow diagram (DFD) of ExpenseTracker
5	Flowchart of the add-transaction process with validation and notification de-duplication
6	Entity-Relationship Diagram (7 entities, all child records keyed by <code>user</code>)
7	Login: split-screen hero, demo-credential hint and JWT-based sign-in
8	Dashboard (light): total balance hero card, month income/expense, six-month chart, category donut, budget status and recent transactions
9	Dashboard (dark mode): identical layout with the dark palette applied through CSS variables
10	Transactions: debounced search, filter panel, favourites toggle, pagination and CSV export
11	Add Transaction modal: income/expense segmented toggle, sanitised amount, category filtered by type, tags and payment method
12	Budgets: month navigation, overall summary and per-category status (On track / Near limit / Exceeded)
13	Analytics with the Bar chart selected and the vertical chart-type selector
14	The same data switched to a Line chart instantly, without any re-fetch
15	Monthly report: month-over-month insight banner, five KPI cards, six-month chart, category change % table and top categories
16	Savings goals with progress bars, "% complete" and the Add-money contribution modal
17	Recurring expenses split into Active and Paused plans, with next due date and pause/resume toggle
18	Categories: expense/income tabs, emoji icon picker and 12-colour swatch picker
19	Profile → Appearance: eight selectable colour themes with live swatches
20	An alternate theme (Cream Burgundy) demonstrating the pluggable theme engine
21	Friendly empty state instead of a blank table (every list page has one, plus loading skeletons)
22	<code>server/src/utils/processRecurring.js</code> — auto-posting of due recurring expenses
23	<code>server/src/utils/notify.js</code> — create-once helper with stable de-duplication keys
24	<code>server/src/controllers/analyticsController.js</code> — dashboard aggregation

25	<code>server/src/controllers/transactionController.js</code> — CSV export endpoint
26	<code>client/src/utils/amount.js</code> — client-side amount sanitization
27	<code>client/src/themes/option8WarmEspressoSage.js</code> — one theme = one separate file
28	<code>client/src/pages/Analytics.jsx</code> — vertical multi-chart selector

Figure numbers and captions are maintained automatically: running `npm run report` renumbers every figure in document order, refreshes this index and fills the page numbers in the table of contents.

1. Introduction & Problem Definition

1.1 Background

Most people know roughly what they earn but not where it goes. Tracking every expense by hand is tedious, and existing mobile apps are often cluttered with advertisements, locked behind subscriptions or require handing personal financial data to a third-party cloud service. For students and salaried individuals there is a need for a simple, private, self-hosted tracker that gives a clear picture of monthly spending within a couple of clicks.

1.2 Problem Statement

There is no lightweight, self-hosted web application that allows a user to record income and expenses quickly, stay within monthly budgets, work towards savings goals, automate repeated payments and see clear analytics — without relying on third-party services or an internet connection beyond the local machine. Manual methods (diary / Excel) do not validate data, cannot warn before a budget is broken, and cannot produce month-over-month comparisons automatically.

1.3 Proposed Solution

ExpenseTracker provides a single dashboard for the complete personal-finance workflow:

- Users register and log in; every record is scoped to that user only.
- Transactions (income / expense) are added with category, payment method, tags, notes, receipt image and favourite flag.
- Per-category monthly budgets show progress and change status automatically (On track → Near limit → Exceeded).
- Savings goals track target versus current amount with contributions and milestone alerts.
- Recurring expenses post themselves as real transactions when they become due.
- Analytics and monthly reports answer "where did the money go?" with charts and percentage change versus last month.
- Notifications warn before budgets break, but the same warning is never repeated (de-duplication).

1.4 Advantages of the Proposed System

Advantage	Description
Single source of truth	All records live in one MongoDB database; no scattered files or sheets.
Automatic insight	Charts, budget status and month-over-month percentages are computed by the server, not by hand.
Automation	Recurring bills post themselves; notifications fire before a budget breaks.
Privacy	Self-hosted; data never leaves the machine. Ownership checks prevent cross-user access.
Data integrity	Amounts are sanitised on the client <i>and</i> the server; schema validation and unique indexes guard the database.
Portability	CSV export moves the filtered data into Excel / Tally in one click.
Usability	Light/dark mode, eight colour themes, loading skeletons, empty states and keyboard-friendly modals.

2. Objectives, Scope & Feasibility

2.1 Objectives

1. Build a full-stack application covering the complete finance workflow: transactions → budgets → goals → reports.
2. Provide secure user accounts where each user can only access their own data (strict per-user scoping).
3. Give meaningful insight through interactive charts (Bar / Line / Area / Pie / Donut) and month-over-month comparison.
4. Warn users automatically when budgets are nearly ($\geq 80\%$) or fully ($\geq 100\%$) used, without repeating the same warning.
5. Offer a clean, consistent and accessible interface with light/dark mode and eight selectable colour themes.
6. Validate every amount on both ends so bad data can never reach the database.
7. Keep the code simple and modular: small single-purpose files and sequential `async/await` flow that is easy to explain, review and maintain.

2.2 Scope of the Project

In scope: authentication (register / login / profile), transaction management with search, filters and pagination, categories, monthly budgets, savings goals, recurring expenses, analytics, monthly reports, notifications, CSV export, receipt upload, favourites, tags, appearance customisation (8 themes + dark mode) and responsive layout.

Out of scope: bank / UPI account integration, multi-user shared or family accounts, native mobile applications, cloud deployment and multi-currency conversion — these are listed in Future Scope (Chapter 14).

2.3 Feasibility Study

Type	Assessment
Technical	All required technologies (Node.js, Express, React, MongoDB, Mongoose, Redux Toolkit, Tailwind CSS) are open-source and free. The complete stack runs on a normal laboratory computer with Node ≥ 18 and a local MongoDB instance. No paid API, cloud account or licence is needed.
Operational	The UI is form-driven and follows a standard dashboard pattern (sidebar, cards, tables, modals). A first-time user can register, add a transaction and read the dashboard without training. Demo accounts with seeded data allow immediate demonstration.
Economic	Development cost is zero (open-source toolchain). Running cost is zero (local MongoDB, no hosting). CSV export replaces any need for a paid reporting tool.
Schedule	Modular design (models → routes → controllers → pages) allowed incremental development and weekly demonstrable increments within the semester timeline.

3. System Analysis — Requirements & Use Cases

3.1 Functional Requirements

FR	Requirement	Description
FR1	User registration & login	Create an account with name, email and password (min 6 chars); log in to receive a JWT valid for 7 days.
FR2	Profile management	Update display name, currency (INR/USD/EUR/GBP/JPY/CAD/AUD) and monthly income target.
FR3	Transaction CRUD	Add, view, edit and delete income/expense records with amount, category, date, description, payment method, tags and notes.
FR4	Search & filter	Debounced text search plus filters for date range, category, type, payment method, amount range and favourites-only; server-side pagination.
FR5	Category management	Create custom categories with emoji icon and colour; separate income and expense types; 19 default categories seeded.
FR6	Monthly budgets	Set a limit per category per month; system reports spent amount, percentage, remaining amount and status.
FR7	Savings goals	Create goals with target amount and date; add money; show progress percentage and milestone alert at 100%.
FR8	Recurring expenses	Define repeated payments (daily/weekly/monthly/yearly) with active/paused toggle; due items auto-post as transactions.
FR9	Analytics	Income vs expense by month, spending by category, payment-method split; switchable chart types without re-fetching.
FR10	Monthly report	Month navigator with month-over-month insight, six-month chart, category change %, top categories and payment breakdown.
FR11	Notifications	Budget warning/exceeded, goal reached, recurring posted and welcome alerts in a bell dropdown with unread badge.
FR12	CSV export	Download the currently filtered transactions (or a selected month) as a date-stamped CSV file.
FR13	Receipt upload	Attach an image (≤5 MB, image MIME types only) to a transaction and preview it in a lightbox.
FR14	Favourites	Star frequent transactions and open a favourites-only list.
FR15	Appearance	Toggle light/dark mode and choose one of eight colour themes; both persisted in localStorage.

3.2 Non-Functional Requirements

Category	Requirement & How it is met
Security	JWT bearer auth, bcrypt hashing (10 rounds), helmet headers, single-origin CORS allow-list, ownership check on every query.
Performance	Compound indexes on <code>user+date</code> , <code>user+month+year+category</code> ; aggregation pipelines run inside MongoDB; debounced search reduces requests; charts animate at 250 ms.
Usability	Consistent component classes, loading skeletons on every page, friendly empty states, Escape-to-close modals, visible focus outlines, reduced-motion support.
Reliability	Central error handler returns a uniform <code>{ success, data message }</code> envelope; 401 interceptor clears the token and redirects to login.
Portability	Cross-platform (Windows/Linux/macOS), browser-based client, environment variables isolated in two config files.
Maintainability	Small single-purpose files (models / controllers / routes / utils / slices), no code duplication of env access, one HTTP client instance.

3.3 Use-Case Diagram

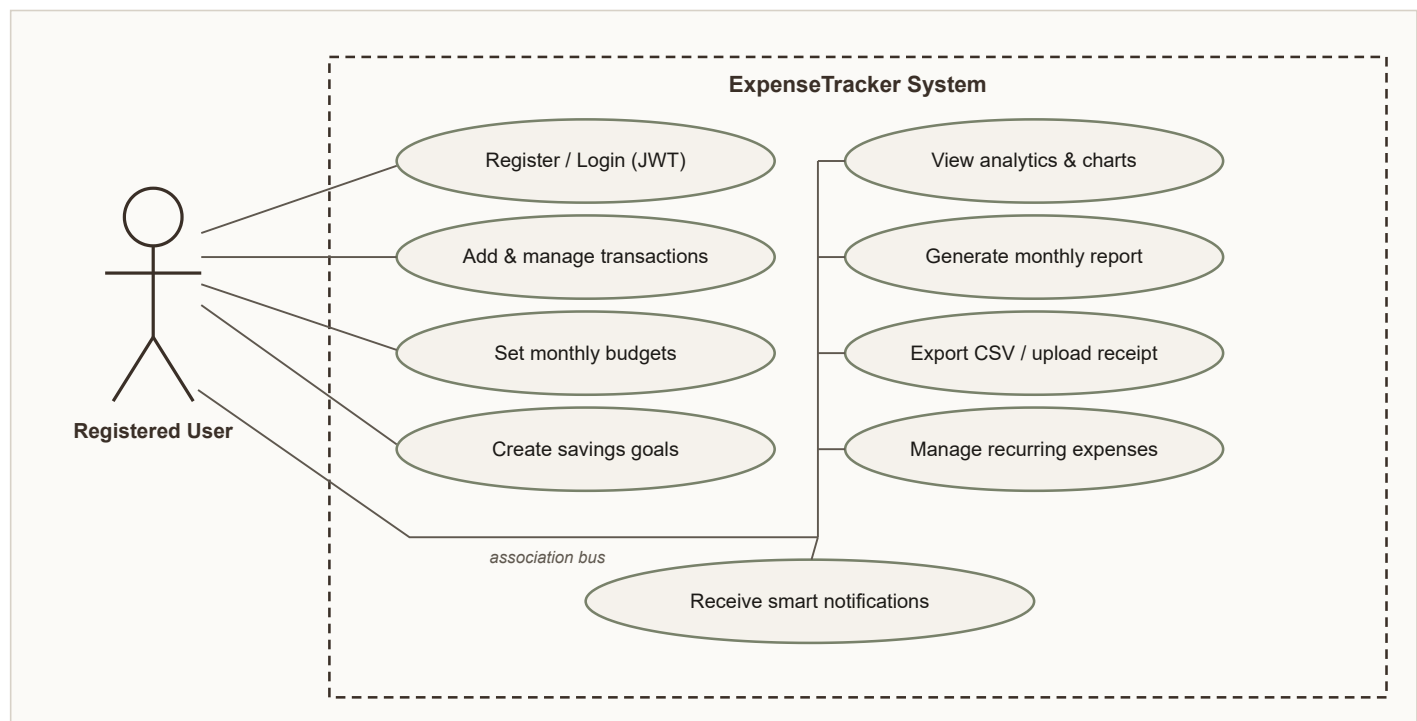


Figure 1 — Use-Case Diagram of ExpenseTracker

4. Technology Stack & Justification

Layer	Technology (version)	Purpose & justification
Database	MongoDB (local, DB <code>expenseTracker</code>)	Schema-flexible document store suits variable finance records (tags, notes); aggregation framework computes reports inside the database.
ODM	Mongoose 8	Declarative schemas, built-in validation, references, timestamps and compound indexes.
Backend	Node.js + Express 4	Lightweight REST API on port 5000; middleware pipeline for auth, uploads and error handling.
Frontend	React 18 (Vite 5)	Component-based SPA on port 5173; fast HMR during development and optimised production build.
State	Redux Toolkit 2 + React Redux 9	Predictable store with eight slices and async thunks; no prop-drilling for server data.
Routing	React Router v6	Protected and public route guards (<code>ProtectedRoute</code> / <code>PublicRoute</code>).
Charts	Recharts 2	Declarative Bar / Line / Area / Pie / Donut charts with responsive containers.
Styling	Tailwind CSS 3.4 + CSS variables	Utility classes mapped to CSS custom properties → theming and dark mode without touching components.
HTTP	Axios	Single instance with request (token attach) and response (401 logout) interceptors.
Auth	jsonwebtoken + bcryptjs	Stateless HS256 tokens (7-day expiry) and salted password hashing (10 rounds).
Uploads	Multer	Disk storage for receipt images, MIME-type filter and size limit.
Security	helmet, CORS, morgan	Secure HTTP headers, single-origin allow-list, request logging in development.
Feedback	react-hot-toast, react-icons	Non-blocking success/error toasts and a consistent icon set.

Why this combination? The MERN stack keeps one language (JavaScript) across the entire application, which let the development focus on features instead of context switching. MongoDB's aggregation pipelines moved all reporting logic server-side, while React + Redux kept the UI reactive. The Vite dev server proxies `/api` and `/uploads` to the backend, so the browser talks to a single origin and CORS stays trivial in development.

5. System Design — Architecture, Data Flow & Structure

5.1 Three-Tier Architecture

ExpenseTracker follows a classic three-tier client–server architecture: a React single-page application (presentation tier), an Express REST API (application tier) and MongoDB (data tier). No page ever talks to the database directly — every read or write passes through the API, which authenticates the token and scopes the query to that user.

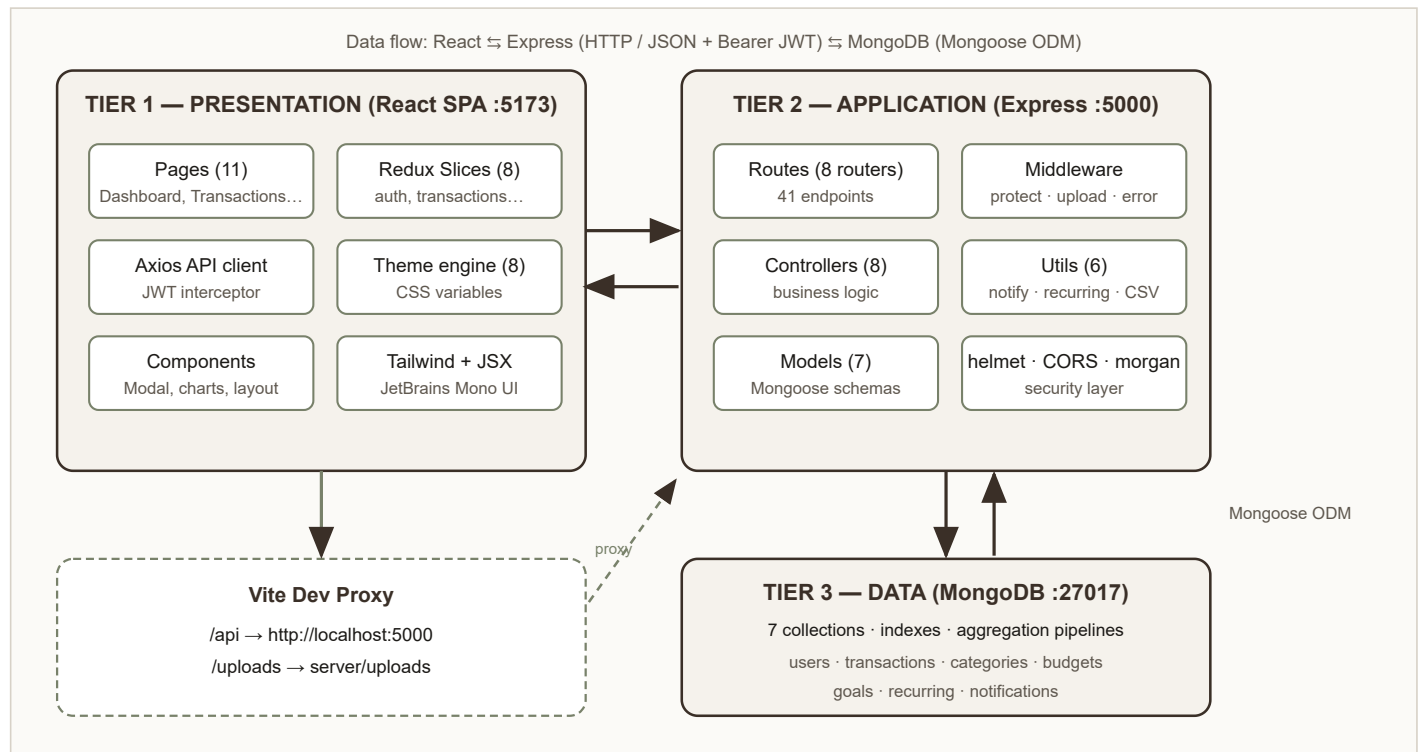


Figure 2 — Three-tier architecture of ExpenseTracker

5.2 Request Lifecycle

1. The page dispatches a Redux thunk, which calls the shared Axios instance in `services/api.js`.
2. The request interceptor attaches `Authorization: Bearer <token>` from `localStorage`.
3. The Express route runs the `protect` middleware, which verifies the JWT and loads `req.user`.
4. The controller validates and sanitises input (for example `toAmount`), then runs the query with `user: req.user._id`.
5. Mongoose executes the query (or aggregation pipeline) against MongoDB and returns documents.
6. The response returns a standardised `{ success, data | message }` envelope.
7. The thunk stores the payload in the slice; components re-render and charts update.
8. On HTTP 401 the response interceptor clears the token and redirects to `/login`; other errors land in the central error handler.

5.3 Context Diagram (DFD Level-0)

The context diagram treats the whole application as a single process and shows only the external entity it interacts with and the data stores it owns. The registered user exchanges credentials and transaction data with the system; the system returns dashboards, reports, alerts and CSV files, and reads/writes all persistent records in MongoDB.

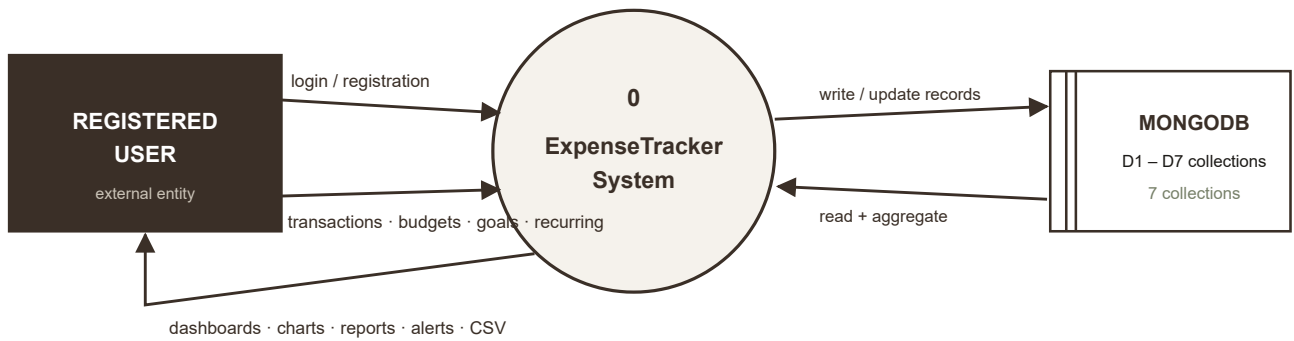


Figure 3 — Context diagram (Level-0 DFD) of ExpenseTracker

5.4 Level-1 Data Flow Diagram

Level 1 decomposes the single process into five sub-processes and names the data stores they read from and write to. All flows originate from or terminate at the registered user; dashed lines show read-only (aggregate) flows.

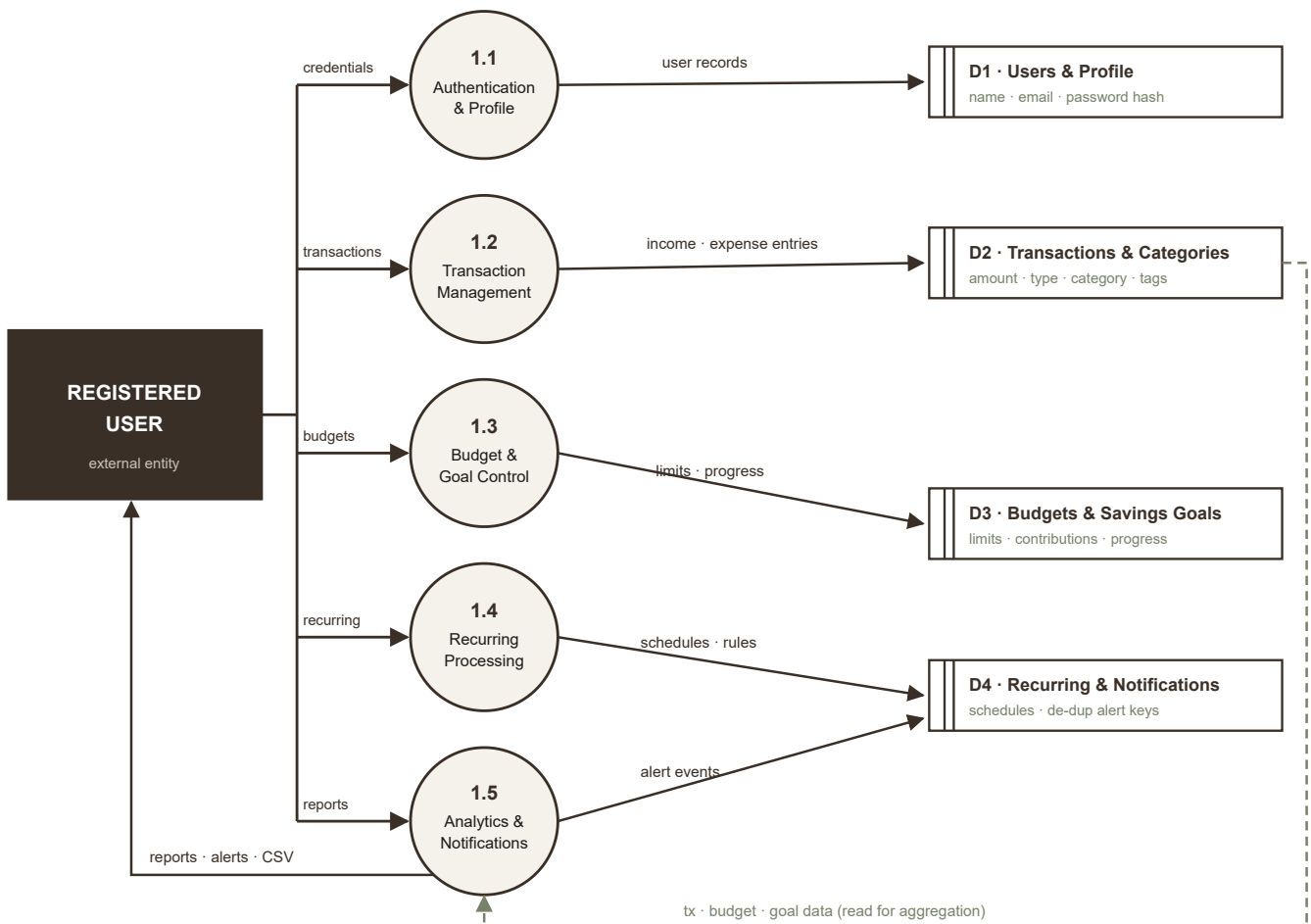


Figure 4 — Level-1 data flow diagram (DFD) of ExpenseTracker

5.5 Flowchart — Adding a Transaction

The flowchart below traces one complete business process: entering a transaction, validating it on the client and on the server, persisting it, applying budget / recurring rules, de-duplicating the resulting notification and updating the UI.

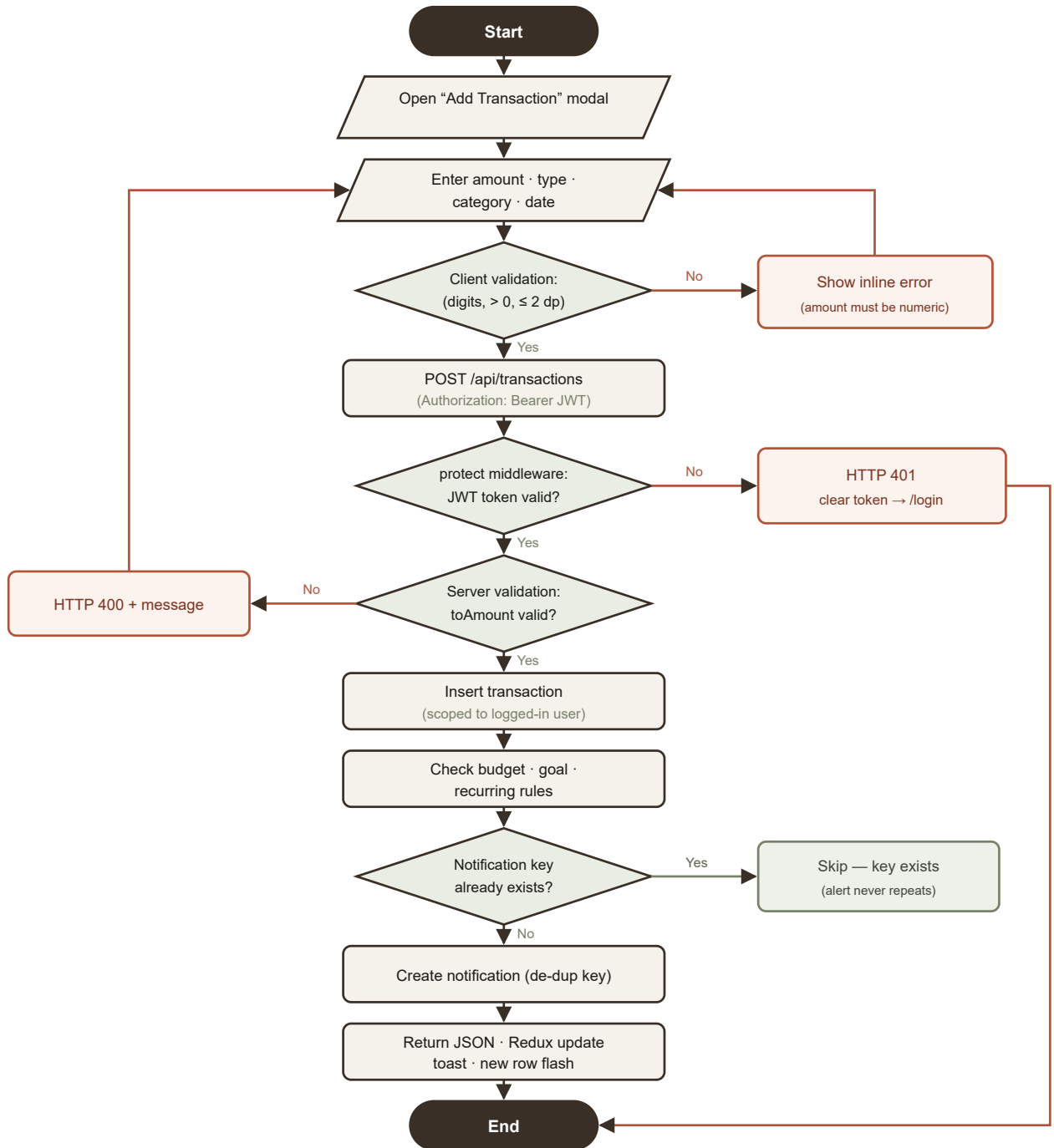


Figure 5 — Flowchart of the add-transaction process with validation and notification de-duplication

5.6 Project Structure

```

ExpenseTracker/
├─ server/                                # Express REST API
│  └─ src/
│     │
│     │ └─ config/env.js                 # ONLY file reading process.env (defaults)
│     │ └─ config/db.js                  # mongoose.connect + exit(1) on failure
│     │ └─ models/                       # 7 models: User, Transaction, Category,
│     │                                     # Budget, SavingsGoal, RecurringExpense, Notification
│     │ └─ middleware/                   # auth.js (protect), error.js, upload.js (Multer)
│     │ └─ utils/                         # sanitize, processRecurring, notify,
│     │                                     # defaultCategories, buildCsv, budgetStatus
│     │ └─ controllers/                  # 8 controllers (~1,200 lines total)
│     │ └─ routes/                       # 8 routers → 40 endpoints (+ /api/health)
│     │ └─ seed/seed.js                  # 2 demo users + realistic data
│     │ └─ uploads/                      # receipt images (git-ignored)
│     └─ app.js, server.js
└─ client/                               # React SPA (Vite)
   └─ src/
      │
      │ └─ config/env.js                 # ONLY file reading import.meta.env
      │ └─ themes/                       # 8 theme files + registry (getTheme/applyTheme)
      │ └─ features/                     # 8 Redux slices + store/index.js
      │ └─ services/api.js               # Axios instance + interceptors
      │ └─ components/                   # Modal, EmptyState, Loader, ProgressBar,
      │                                     # charts/MonthlyChart, layout/{Sidebar,Navbar,MainLayout}
      │ └─ pages/                        # 11 pages (Dashboard ... Profile)
      │ └─ hooks/useDebounce.js
      │ └─ utils/                        # amount.js, format.js, constants.js
      └─ index.css                      # CSS variables + component classes

```

6. Database Design — ER Diagram & Schema

6.1 Entity-Relationship Diagram

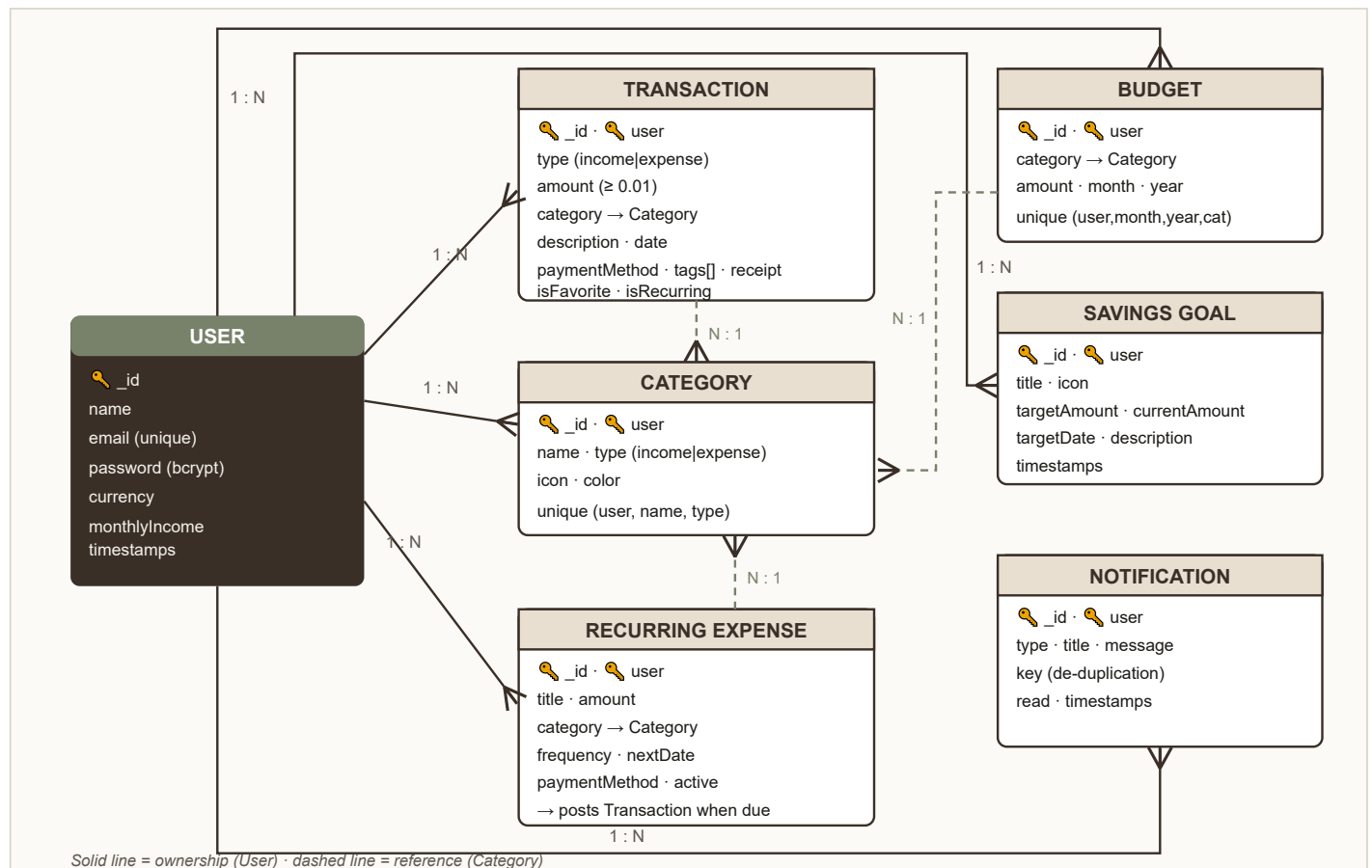


Figure 6 — Entity-Relationship Diagram (7 entities, all child records keyed by `user`)

6.2 Schema Definitions

Collection	Key fields (type / constraint)	Indexes & notes
User	<code>name</code> (2–50 chars), <code>email</code> (unique, regex-validated, lowercase), <code>password</code> (min 6, <code>select:false</code> , bcrypt-hashed in a <code>pre('save')</code> hook), <code>profileImage</code> , <code>currency</code> (enum of 7, default INR), <code>monthlyIncome</code> (min 0)	Unique index on <code>email</code> ; instance methods <code>matchPassword()</code> and <code>getSignedJwtToken()</code> .
Transaction	<code>user</code> (ref), <code>type</code> (income expense), <code>amount</code> (min 0.01), <code>category</code> (ref, required), <code>description</code> (≤200), <code>date</code> , <code>paymentMethod</code> (7-value enum), <code>tags[]</code> , <code>notes</code> (≤500), <code>receipt</code> , <code>isFavorite</code> , <code>isRecurring</code> , <code>recurringExpense</code> (ref)	<code>{user:1,date:-1}</code> , <code>{user:1,type:1}</code> , <code>{user:1,category:1}</code>
Category	<code>user</code> , <code>name</code> (≤50), <code>type</code> (income expense), <code>icon</code> (emoji), <code>color</code> (hex)	Unique compound <code>{user,name,type}</code> — one name can exist once per type per user.
Budget	<code>user</code> , <code>category</code> (ref), <code>amount</code> (min 0), <code>month</code> (1–12), <code>year</code>	Unique compound <code>{user,month,year,category}</code> — exactly one budget per category per month.
SavingsGoal	<code>user</code> , <code>title</code> (≤100), <code>targetAmount</code> (min 1), <code>currentAmount</code> (min 0, default 0), <code>targetDate</code> , <code>description</code> , <code>icon</code>	Timestamps; progress computed at read time.
RecurringExpense	<code>user</code> , <code>title</code> , <code>amount</code> , <code>category</code> (ref), <code>frequency</code> (daily weekly monthly yearly), <code>nextDate</code> , <code>paymentMethod</code> , <code>active</code> (default true)	When <code>nextDate ≤ today</code> , the worker posts a linked Transaction and advances the date.
Notification	<code>user</code> , <code>title</code> , <code>message</code> , <code>type</code> (budget_warning budget_exceeded goal_progress recurring_due info), <code>key</code> , <code>read</code>	<code>key</code> is the de-duplication key checked before insert.

Design decision: categories are *referenced*, not copied into transactions. Renaming a category instantly updates every historical view of it, and the analytics pipeline uses `$lookup` to join name, icon and colour at read time. Every query in every controller includes `req.user._id`, so users can never see each other's data.

#	Module	Detailed description
1	Authentication	Register (name ≥ 2 chars, valid email, password ≥ 6 , confirm-password check on the client), login, <code>GET /me</code> hydration on every app load, profile update (name, currency, monthly income target). JWT is stored in <code>localStorage</code> and attached automatically to every request.
2	Dashboard	One call to <code>GET /analytics/dashboard</code> returns balance hero card (total balance, all income, all expense, this month), month income/expense cards, six-month bar chart, category donut with top-4 percentages, budget status bars and recent transactions. Shows a time-based greeting and a loading skeleton grid while fetching.
3	Transactions	Add/edit/delete records; debounced search (300 ms); filter panel for type, category, payment method, date range and favourites-only with an active-filter count badge; pagination (10/page); receipt upload, favourite star, edit and delete per row; a new row flashes for 700 ms; deep link <code>?new=1</code> auto-opens the add modal.
4	Categories	Expense/Income tab switch, card grid with emoji icon on a tinted background, 20-icon picker and 12-colour swatch picker in the modal. Deleting a category that a recurring expense still uses is blocked with a clear message.
5	Budgets	Month/year navigation; overall summary bar (spent vs total budget); per-category cards with status label — <i>On track</i> , <i>Near limit</i> ($\geq 80\%$) or <i>Exceeded</i> ($\geq 100\%$) — spent-of-budget, progress bar and remaining/over amount. The add modal only offers expense categories not already budgeted for that month.
6	Savings Goals	Card grid with emoji, title, target date, current-of-target amounts, progress bar and "% complete"; <i>Add money</i> modal for contributions; edit/delete; a notification fires once when a goal reaches 100%.
7	Recurring Expenses	Info banner explaining auto-posting; items split into <i>Active</i> and <i>Paused</i> sections with counts; cards show category icon, title, frequency, next date, amount, payment method and a pause/resume toggle. Due items post automatically as transactions.
8	Analytics	Three yearly KPI cards (income, expenses, savings), a vertical chart-type selector (Bar / Line / Area / Donut / Pie) that swaps the chart instantly without re-fetching, spending breakdown per category with percentages, and a payment-method list with totals and counts.
9	Monthly Reports	Month navigator; insight banner ("You spent 48% less than last month"); five KPI cards each with "% vs last"; six-month income/expense chart; category donut; category analysis table (this month, last month, change %, share); top spending categories ranked bars; payment-method breakdown; month-scoped CSV export.
10	Notifications	Budget warning ($\geq 80\%$), budget exceeded ($\geq 100\%$), goal reached, recurring posted and welcome alerts. Every alert carries a stable key so the same warning never repeats within a month. Bell dropdown polls every 60 s with an unread badge, "Mark all read" and per-item read.
11	CSV Export	Applies the current filters (or a selected month) server-side and downloads a date-stamped CSV built by the <code>buildCsv</code> helper with proper quote escaping.
12	Receipts & Favourites	Attach an image to a transaction with a lightbox preview; star frequent transactions and open a favourites-only list.

13	Appearance	Light/dark mode toggle in the navbar and eight selectable colour themes under Profile → Appearance, both persisted in <code>localStorage</code> and re-applied on every load.
14	UX Polish	Loading skeletons on every page, friendly empty states, 150–250 ms CSS micro-interactions, Escape-to-close modals, visible focus outlines and <code>prefers-reduced-motion</code> support.
15	Validation	Amount sanitization while typing (digits only, one dot, max two decimals) mirrored by a server-side <code>toAmount</code> check that rejects invalid values with HTTP 400.

8. API Reference

The application exposes **41 endpoints** (40 in routers + 1 health check). Every route except register, login and health sits behind the `protect` middleware — **38 of 41 endpoints are authenticated**. All responses share the same `{ success, data | message }` envelope, which keeps front-end error handling uniform.

Router	Method	Endpoint & purpose
/api/auth	POST	/register — create account, returns JWT
	POST	/login — verify credentials, returns JWT
	GET	/me — current user profile (<i>protected</i>)
	PUT	/profile — update name / currency / monthly income (<i>protected</i>)
/api/transactions	GET	/ — list with search, filters and pagination
	POST	/ — create transaction
	GET	/:id — fetch one transaction
	PUT	/:id — update transaction
	DELETE	/:id — delete transaction (also removes receipt file)
	GET	/csv — filtered CSV export
	GET	/favorites — favourites-only list
	PATCH	/:id/favorite — toggle favourite
	PATCH	/:id/receipt — upload receipt image (Multer)
/api/categories	GET	/ — list categories (optional type filter)
	POST	/ — create category
	PUT	/:id — update category
	DELETE	/:id — delete (blocked if a recurring expense uses it)
/api/budgets	GET	/ — budgets for month&year with computed status summary
	POST	/ — create budget
	PUT	/:id — update budget
	DELETE	/:id — delete budget
/api/goals	GET	/ — list savings goals
	POST	/ — create goal
	PUT	/:id — update goal
	DELETE	/:id — delete goal
	POST	/:id/add-money — contribute to a goal
/api/recurring-expenses	GET	/ — list recurring plans

	POST	/ — create plan
	PUT	/:id — update plan
	DELETE	/:id — delete plan
	PATCH	/:id/toggle — pause / resume
/api/analytics	GET	/dashboard — summary, series, category breakdown, budget status, recent tx
	GET	/monthly — income vs expense by month for a year
	GET	/categories — spending by category for a month
	GET	/payment-methods — payment split for a month
	GET	/reports — month-over-month report payload
/api/notifications	GET	/ — list notifications + unread count
	PATCH	/read-all — mark all read
	PATCH	/:id/read — mark one read
	DELETE	/:id — delete one
/api/health	GET	/ — liveness check (public, used during testing)

9. User Interface, Design System & Screenshots

9.1 Design System — Option 8 (Warm White + Espresso + Sage)

The default theme uses a warm white background with espresso brown and sage green accents. All colours are CSS variables (`--bg` , `--text` , `--card` , `--border` , `--primary` , `--accent` , `--sage` , `--beige` , ...) defined for both light and dark mode, so changing theme or mode only swaps variables — no component ever contains colour logic. The full interface uses the **JetBrains Mono** typeface for a distinctive engineering-first look, with a subtle 3.5% fractal-noise paper texture on the background.

Token	Hex	Usage
Background (<code>--bg</code>)	#FAF8F3	Page background
Text (<code>--text</code>)	#29231F	Primary text
Primary — Espresso	#493B32	Buttons, active states, sidebar brand
Accent — Sage	#78816A	Charts, links, highlights
Sage tint (<code>--sage</code>)	#E9EDE4	Sidebar, soft panels, chips
Beige tint (<code>--beige</code>)	#E9DFD1	Balance and stat cards
Card (<code>--card</code>)	#FFFFFF	Surfaces in light mode

9.2 Theme Engine & Dark Mode

Each of the eight themes lives in its own file under `client/src/themes/` — Warm Ivory Forest, Off-White Navy, Cream Burgundy, White Terracotta, White Olive, Pearl Plum, White Cobalt and **Warm Espresso Sage** (default). A small registry exposes `getTheme(id)` and `applyTheme(theme, dark)` , which writes twelve CSS custom properties on `:root` . The choice is made under Profile → Appearance and persisted in `localStorage` together with the dark-mode flag; `App.jsx` re-applies both on every load. Tailwind is configured with `darkMode: 'class'` and colour names that map directly to those variables.

9.3 Interaction & Accessibility

Transitions (150–300 ms) are pure CSS: hover lift on buttons and cards, modal fade+scale (200 ms), progress-bar fill (300 ms), skeleton shimmer and a 700 ms flash highlight on newly added rows. Modals close with the Escape key, inputs are labelled, and focus outlines use a 2 px accent ring. Amount inputs use `inputMode="decimal"` and hide the browser number spinners. Every page has a loading skeleton and a friendly empty state instead of a blank table, and `prefers-reduced-motion` collapses all animation durations to 0.01 ms.

9.4 Screenshots (Category-wise)

All 15 captured screens are grouped below by module category. Each figure is the actual running application (seeded demo account, desktop viewport, light mode unless stated).

9.4.1 Authentication

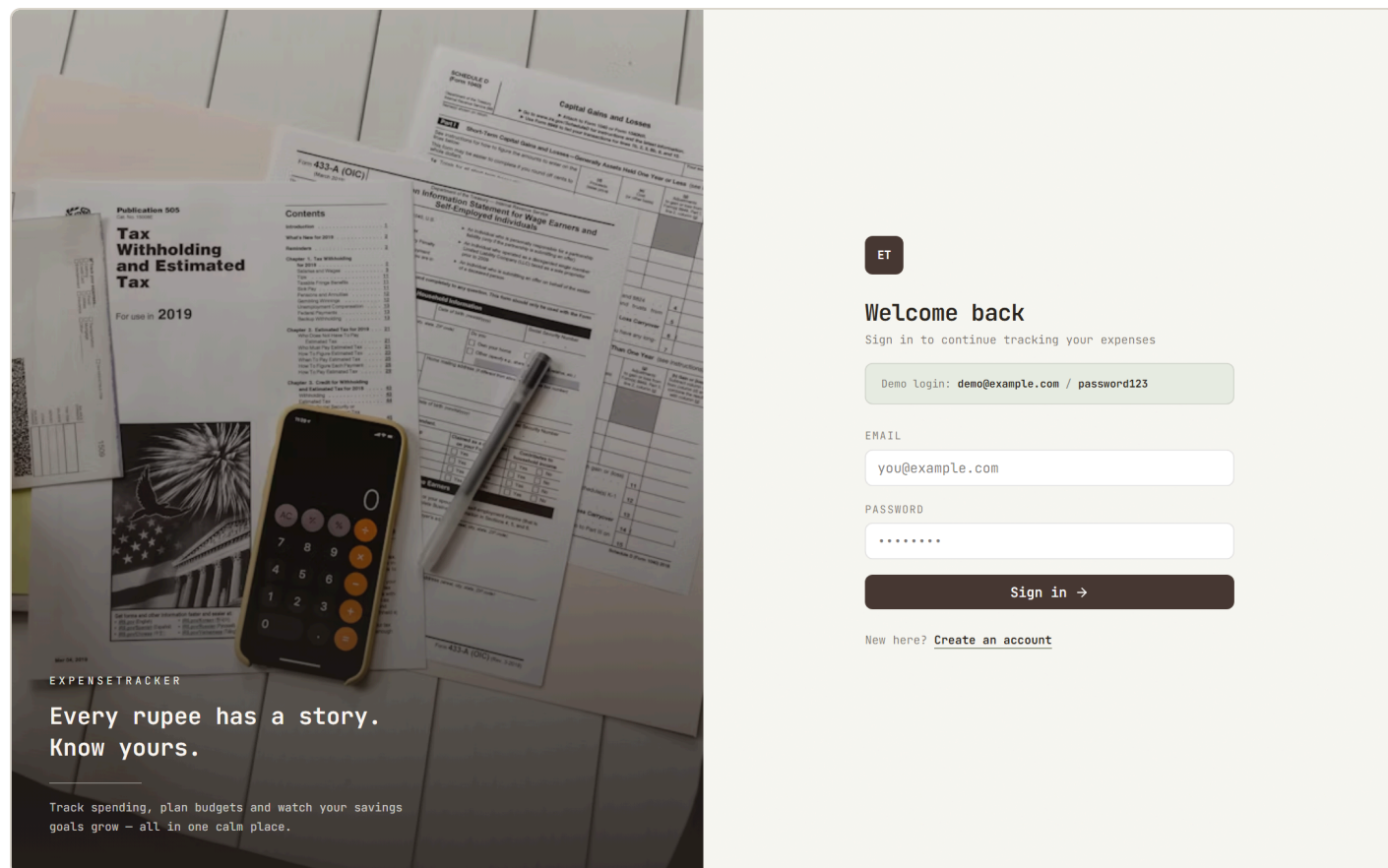


Figure 7 — Login: split-screen hero, demo-credential hint and JWT-based sign-in

9.4.2 Dashboard

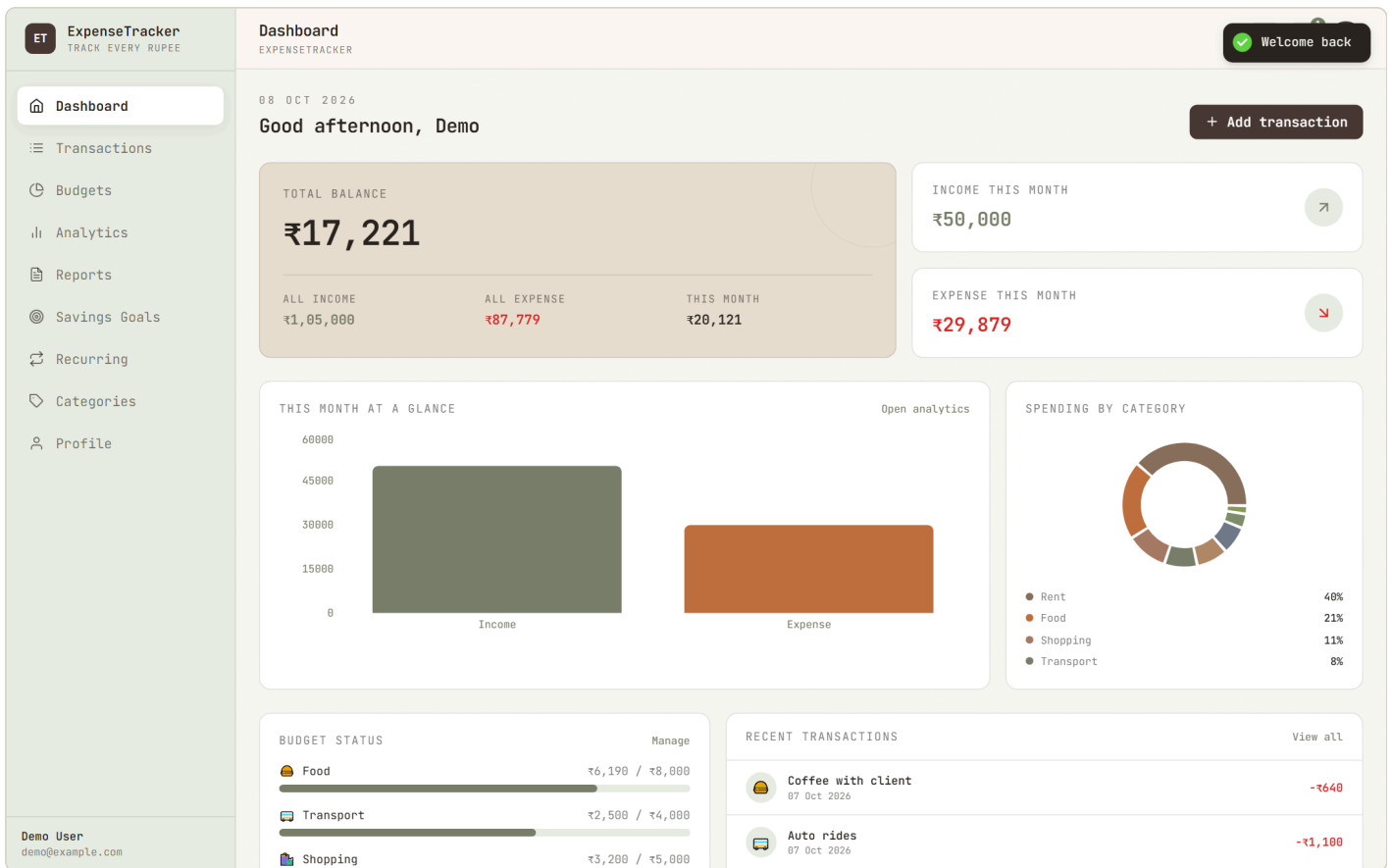


Figure 8 — Dashboard (light): total balance hero card, month income/expense, six-month chart, category donut, budget status and recent transactions

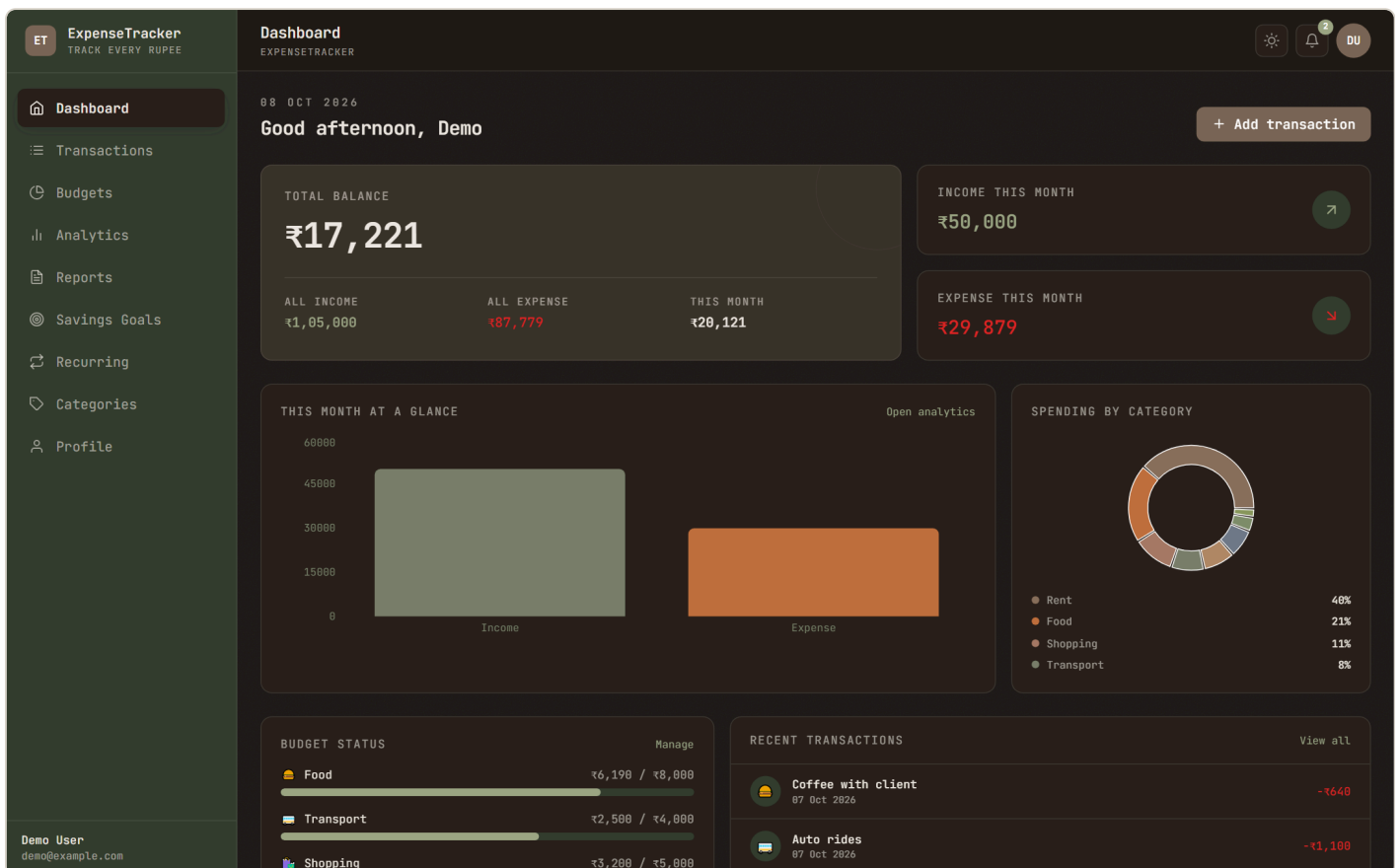


Figure 9 — Dashboard (dark mode): identical layout with the dark palette applied through CSS variables

9.4.3 Transactions

ETExpenseTracker
TRACK EVERY RUPEE

Dashboard

Transactions

Budgets

Analytics

Reports

Savings Goals

Recurring

Categories

Profile

Demo User
demo@example.com

Transactions
EXPENSETRACKER

32 RECORDS

All transactions

Export CSVAdd transaction

Search descriptions...

Filters

Favorites

DATE	DESCRIPTION	CATEGORY	PAYMENT	AMOUNT	ACTIONS
07 Oct 2026	Coffee with client coffee work	Food	Cash	-₹640	⬆️ ⬆️ ✎️ 🗑️
07 Oct 2026	Auto rides commute	Transport	Cash	-₹1,100	⬆️ ⬆️ ✎️ 🗑️
06 Oct 2026	Gym supplements	Health	UPI	-₹990	⬆️ ⬆️ ✎️ 🗑️
06 Oct 2026	Running shoes fitness	Shopping	Credit Card	-₹3,200	⬆️ ⬆️ ✎️ 🗑️
05 Oct 2026	Team lunch work	Food	Credit Card	-₹1,750	⬆️ ⬆️ ✎️ 🗑️
05 Oct 2026	Streaming + games	Entertainment	Credit Card	-₹2,400	⬆️ ⬆️ ✎️ 🗑️
05 Oct 2026	Mobile Plan	Mobile	UPI	-₹499	⬆️ ⬆️ ✎️ 🗑️
04 Oct 2026	Phone bill	Bills	Net Banking	-₹2,100	⬆️ ⬆️ ✎️ 🗑️
03 Oct 2026	Metro card top-up commute	Transport	UPI	-₹1,400	⬆️ ⬆️ ✎️ 🗑️
02 Oct 2026	Monthly groceries	Food	Debit Card	-₹3,800	⬆️ ⬆️ ✎️ 🗑️

Figure 10 — Transactions: debounced search, filter panel, favourites toggle, pagination and CSV export

ETExpenseTracker
TRACK EVERY RUPEE

Dashboard

Transactions

Budgets

Analytics

Reports

Savings Goals

Recurring

Categories

Profile

Demo User
demo@example.com

Transactions
EXPENSETRACKER

32 RECORDS

All transactions

Export CSVAdd transaction

Search descriptions...

Filters

Favorites

DATE	DESCRIPTION	CATEGORY	PAYMENT	AMOUNT	ACTIONS
07 Oct 2026	Coffee with client coffee work	Food	Cash	-₹640	⬆️ ⬆️ ✎️ 🗑️
07 Oct 2026	Auto rides commute	Transport	Cash	-₹1,100	⬆️ ⬆️ ✎️ 🗑️
06 Oct 2026	Gym supplements	Health	UPI	-₹990	⬆️ ⬆️ ✎️ 🗑️
06 Oct 2026	Running shoes fitness	Shopping	Credit Card	-₹3,200	⬆️ ⬆️ ✎️ 🗑️
05 Oct 2026	Team lunch work	Food	Credit Card	-₹1,750	⬆️ ⬆️ ✎️ 🗑️
05 Oct 2026	Streaming + games	Entertainment	Credit Card	-₹2,400	⬆️ ⬆️ ✎️ 🗑️
05 Oct 2026	Mobile Plan	Mobile	UPI	-₹499	⬆️ ⬆️ ✎️ 🗑️
04 Oct 2026	Phone bill	Bills	Net Banking	-₹2,100	⬆️ ⬆️ ✎️ 🗑️
03 Oct 2026	Metro card top-up commute	Transport	UPI	-₹1,400	⬆️ ⬆️ ✎️ 🗑️
02 Oct 2026	Monthly groceries	Food	Debit Card	-₹3,800	⬆️ ⬆️ ✎️ 🗑️

ADD TRANSACTION

ExpenseIncome

AMOUNTDATE

010/08/2026

CATEGORY

Select category

DESCRIPTION

What was it for?

PAYMENT METHODTAGS (COMMA SEPARATED)

Cashhome, work

NOTES

Optional notes

CancelAdd transaction

Figure 11 — Add Transaction modal: income/expense segmented toggle, sanitised amount, category filtered by type, tags and payment method

9.4.4 Budgets

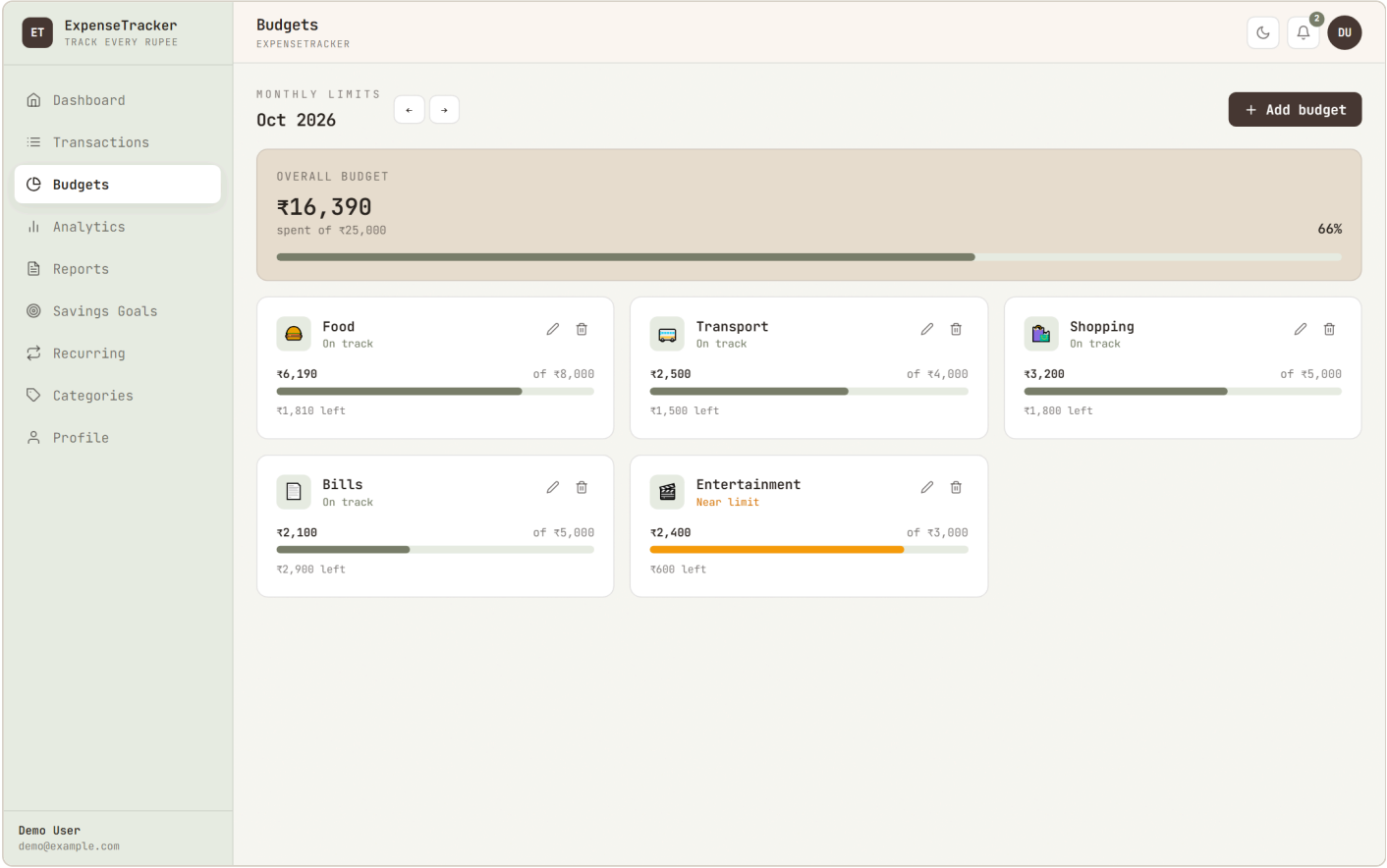


Figure 12 — Budgets: month navigation, overall summary and per-category status (On track / Near limit / Exceeded)

9.4.5 Analytics & Charts

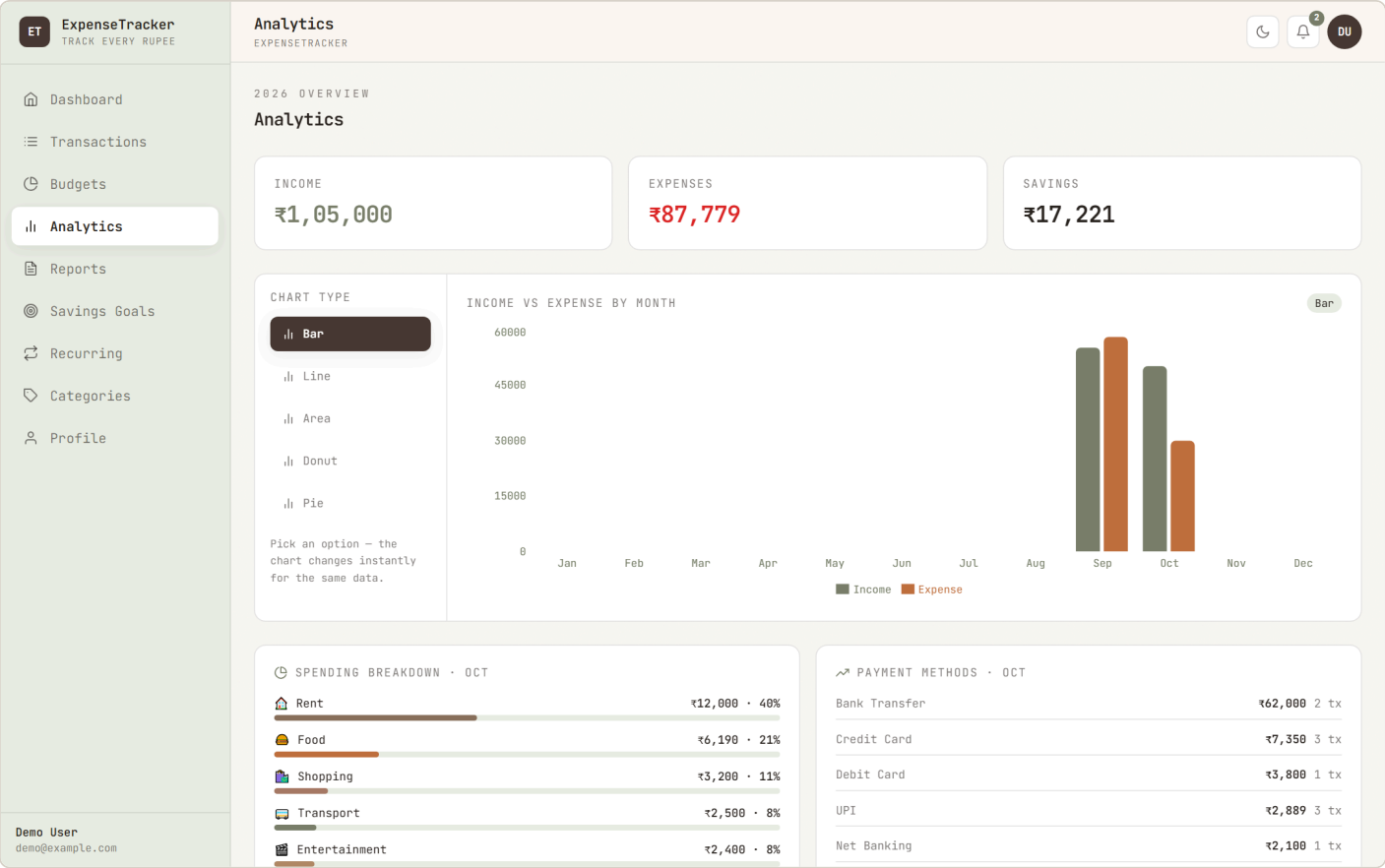


Figure 13 — Analytics with the Bar chart selected and the vertical chart-type selector

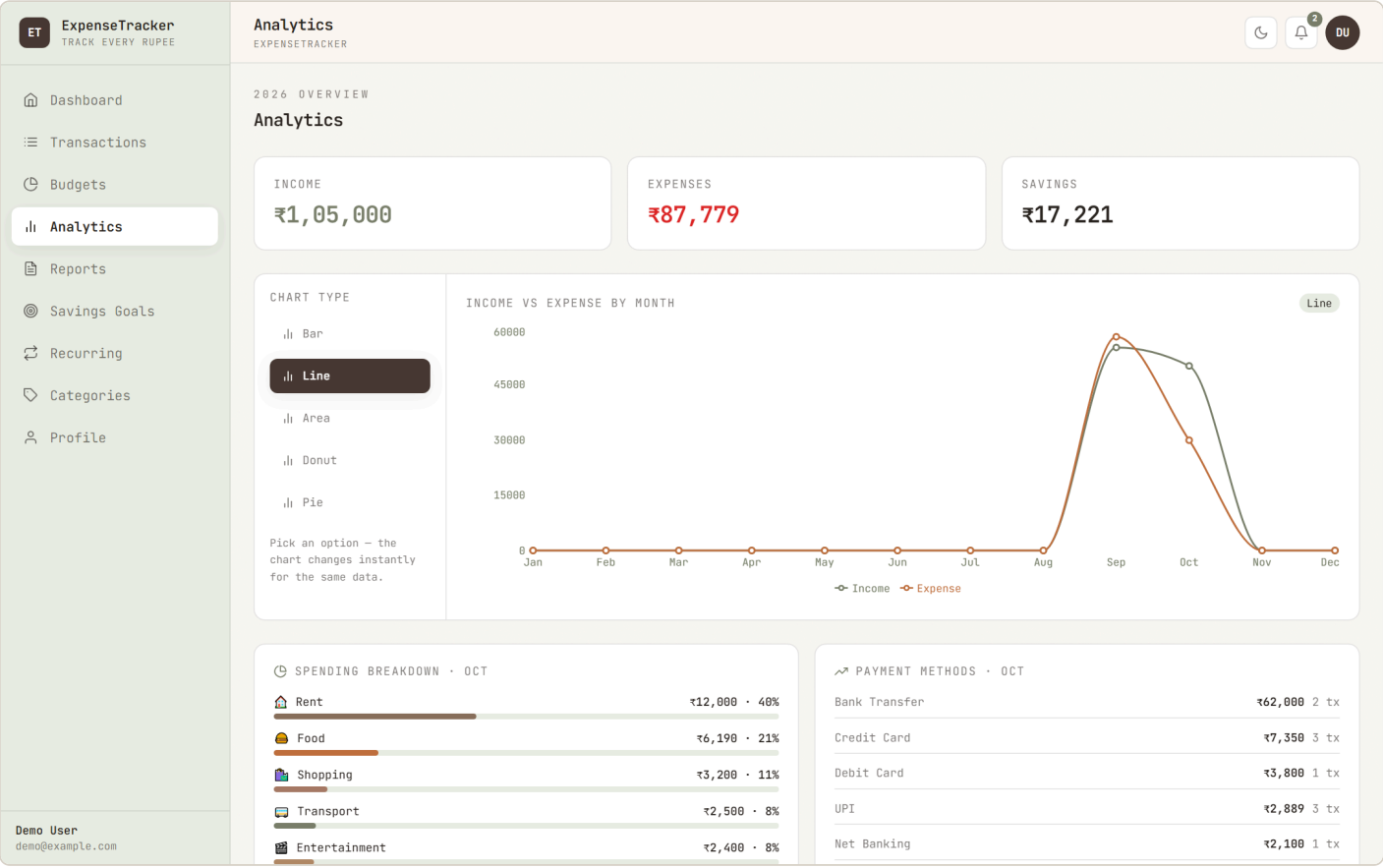
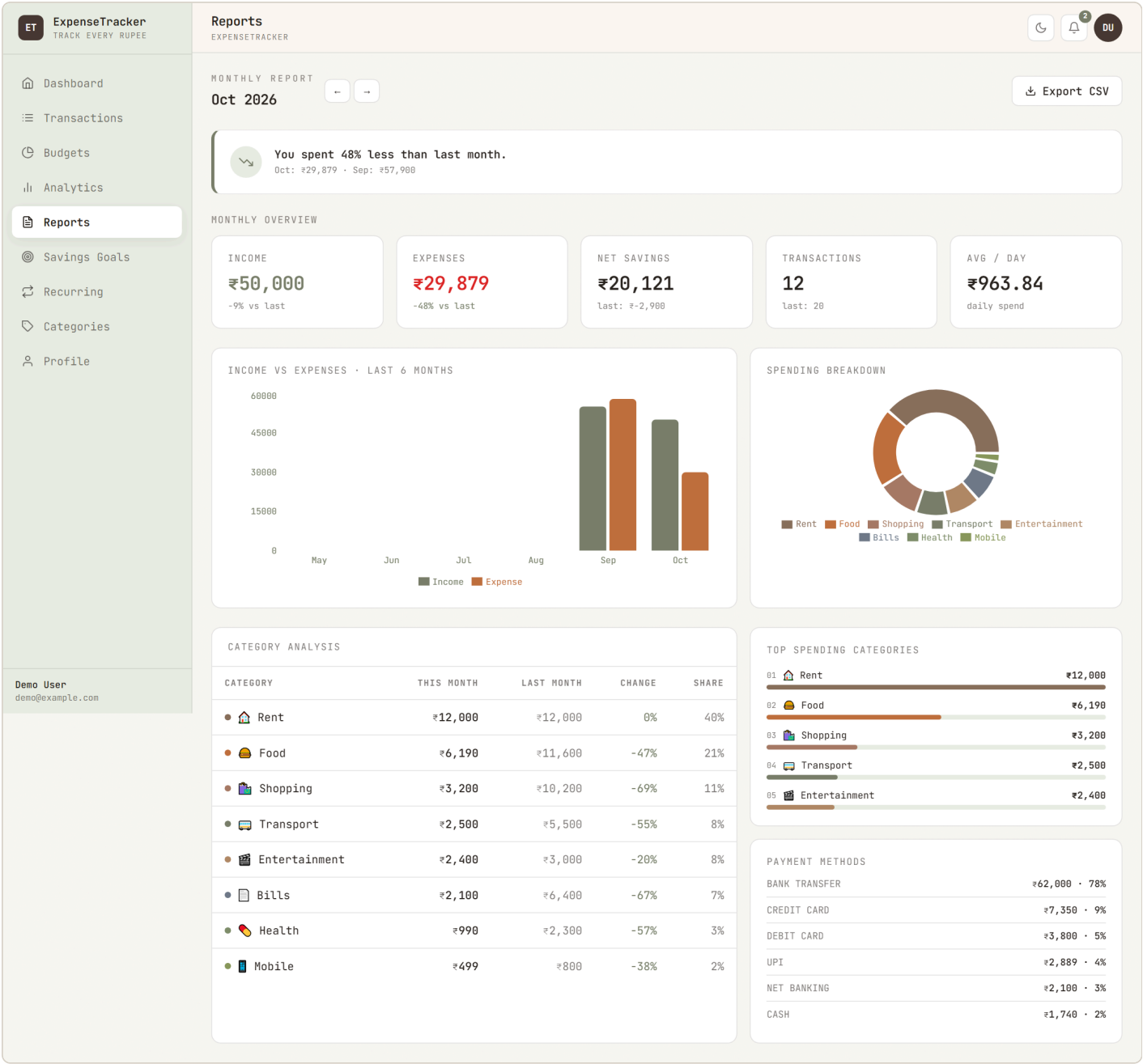


Figure 14 — The same data switched to a Line chart instantly, without any re-fetch

9.4.6 Monthly Reports



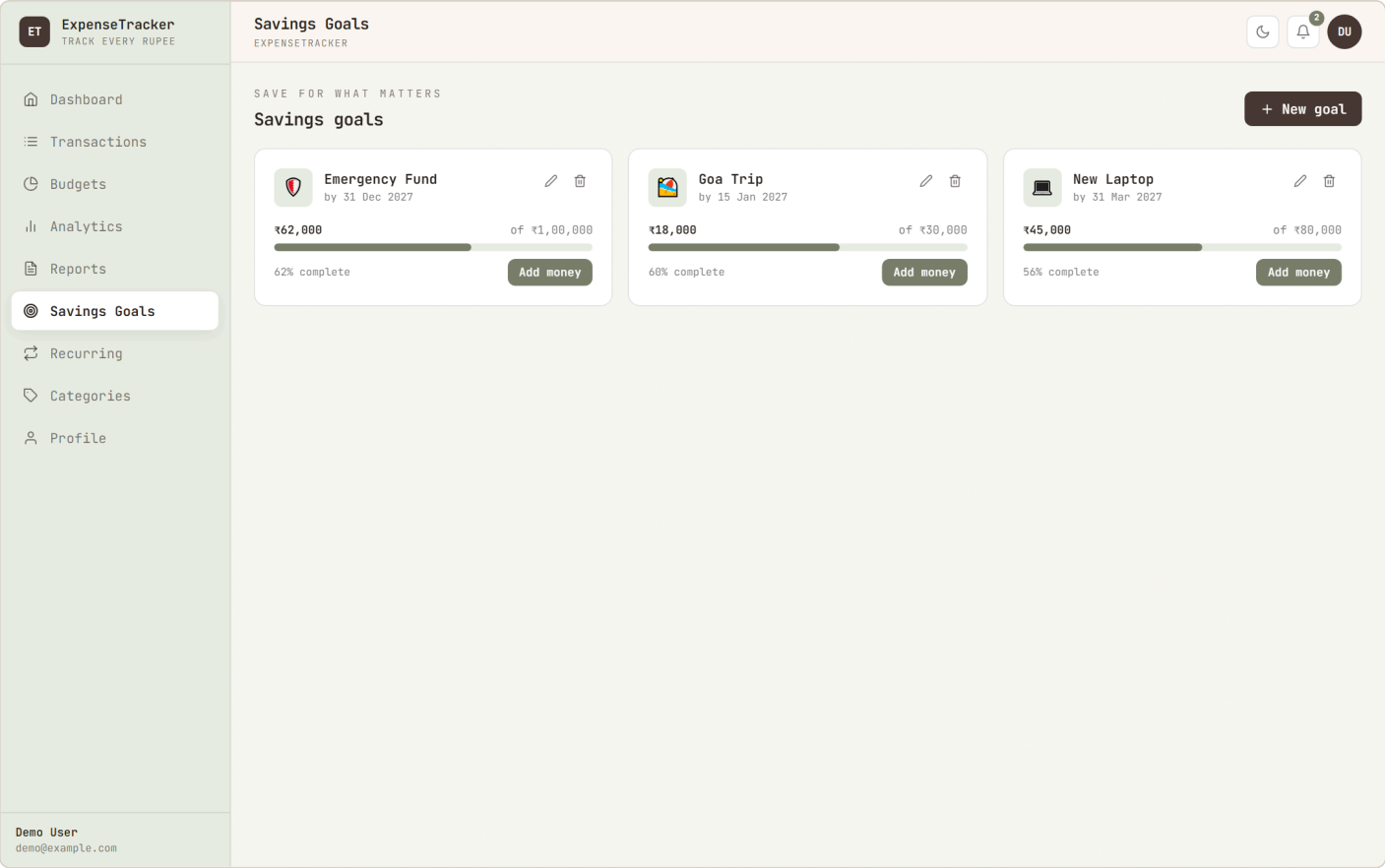


Figure 16 — Savings goals with progress bars, "% complete" and the Add-money contribution modal

9.4.8 Recurring Expenses

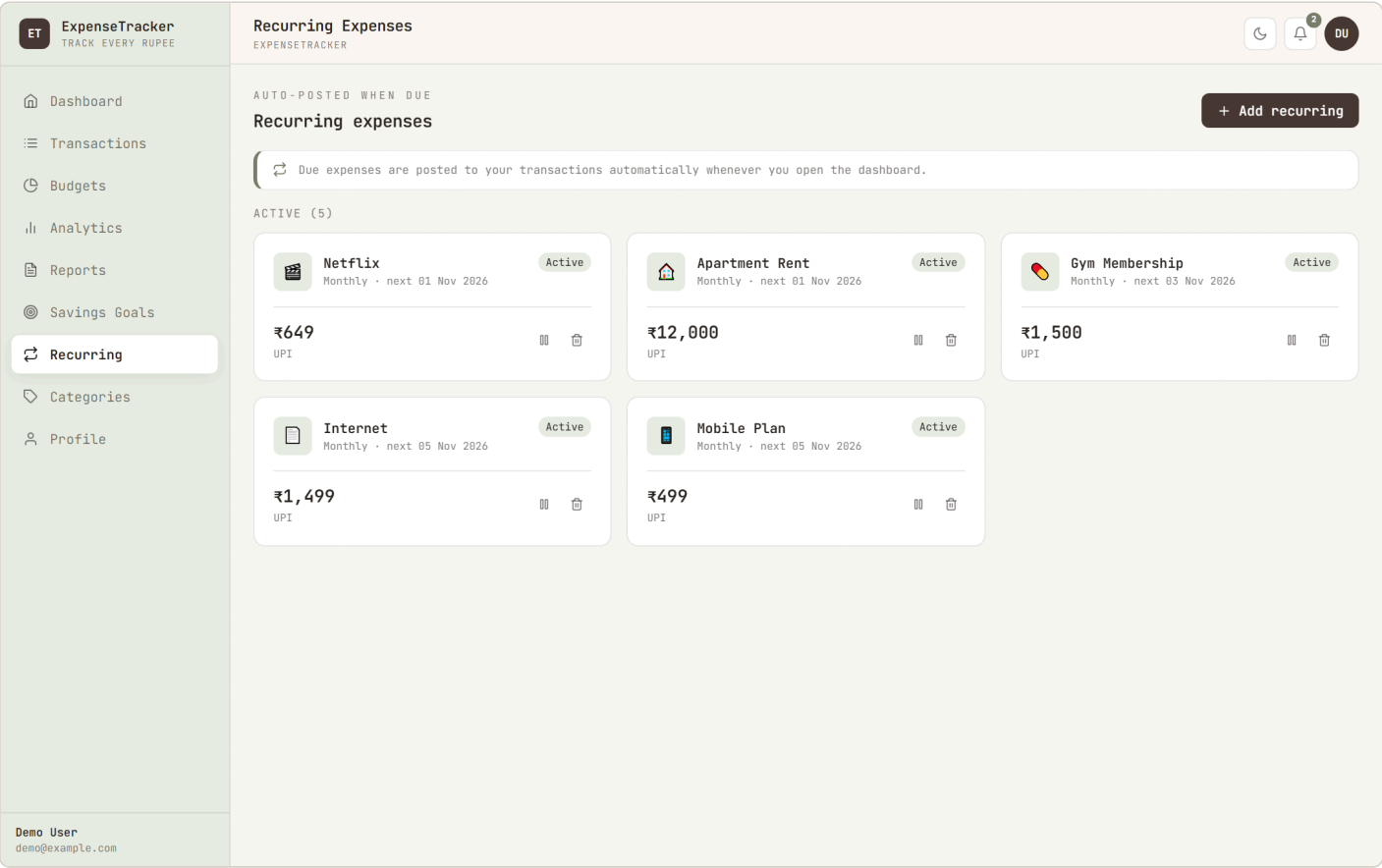


Figure 17 — Recurring expenses split into Active and Paused plans, with next due date and pause/resume toggle

9.4.9 Categories

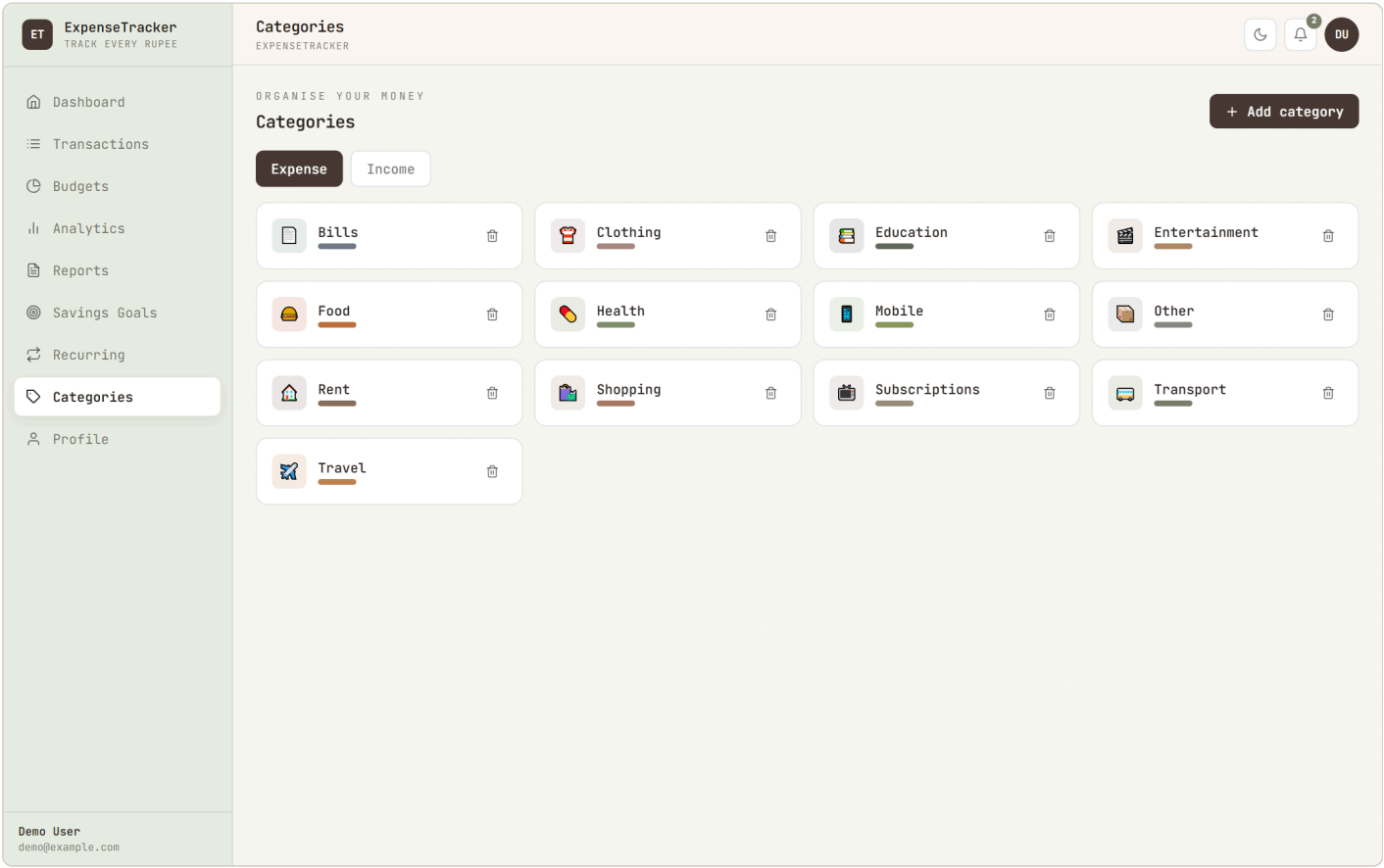


Figure 18 — Categories: expense/income tabs, emoji icon picker and 12-colour swatch picker

9.4.10 Appearance & Themes

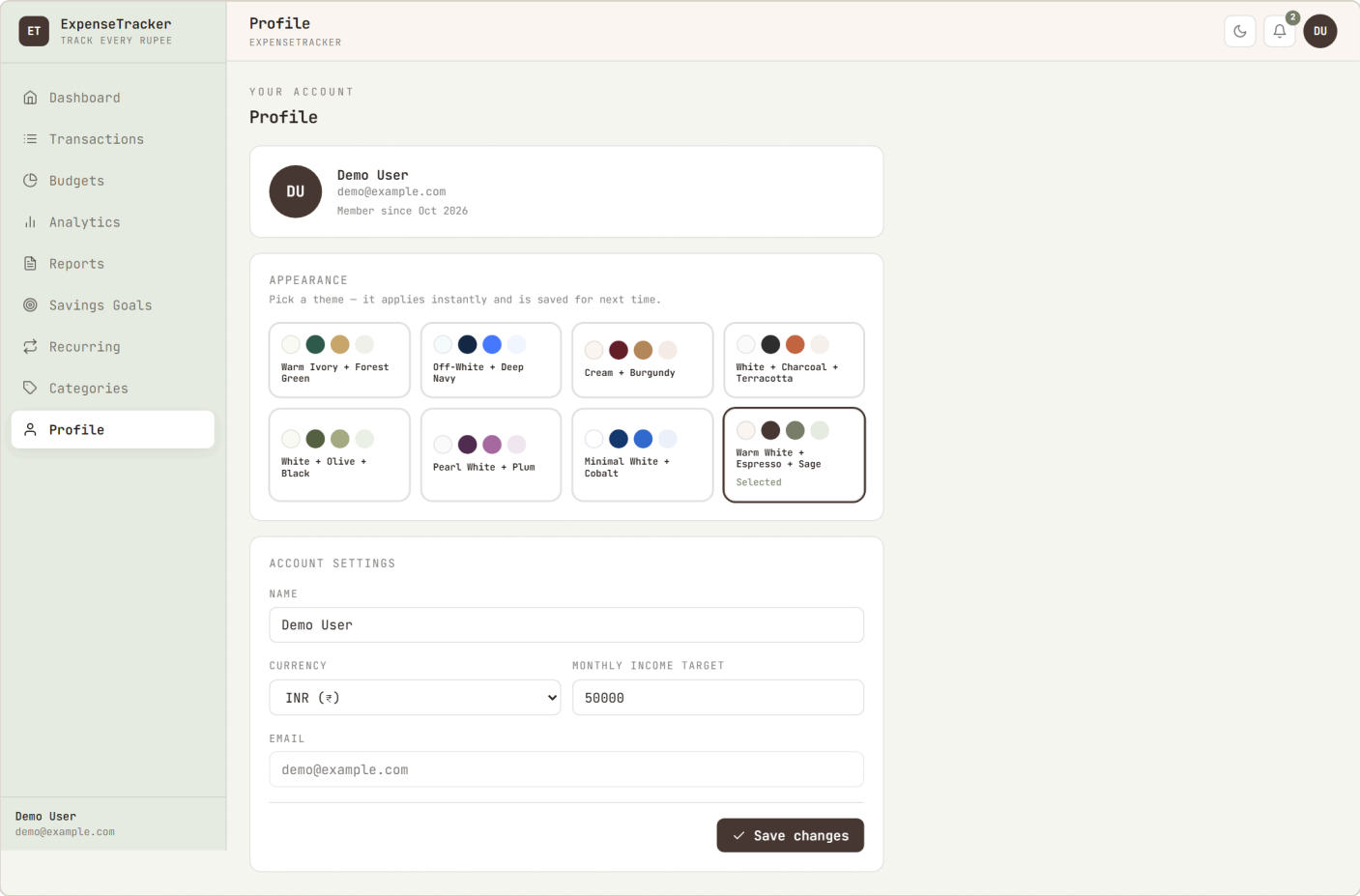


Figure 19 — Profile → Appearance: eight selectable colour themes with live swatches

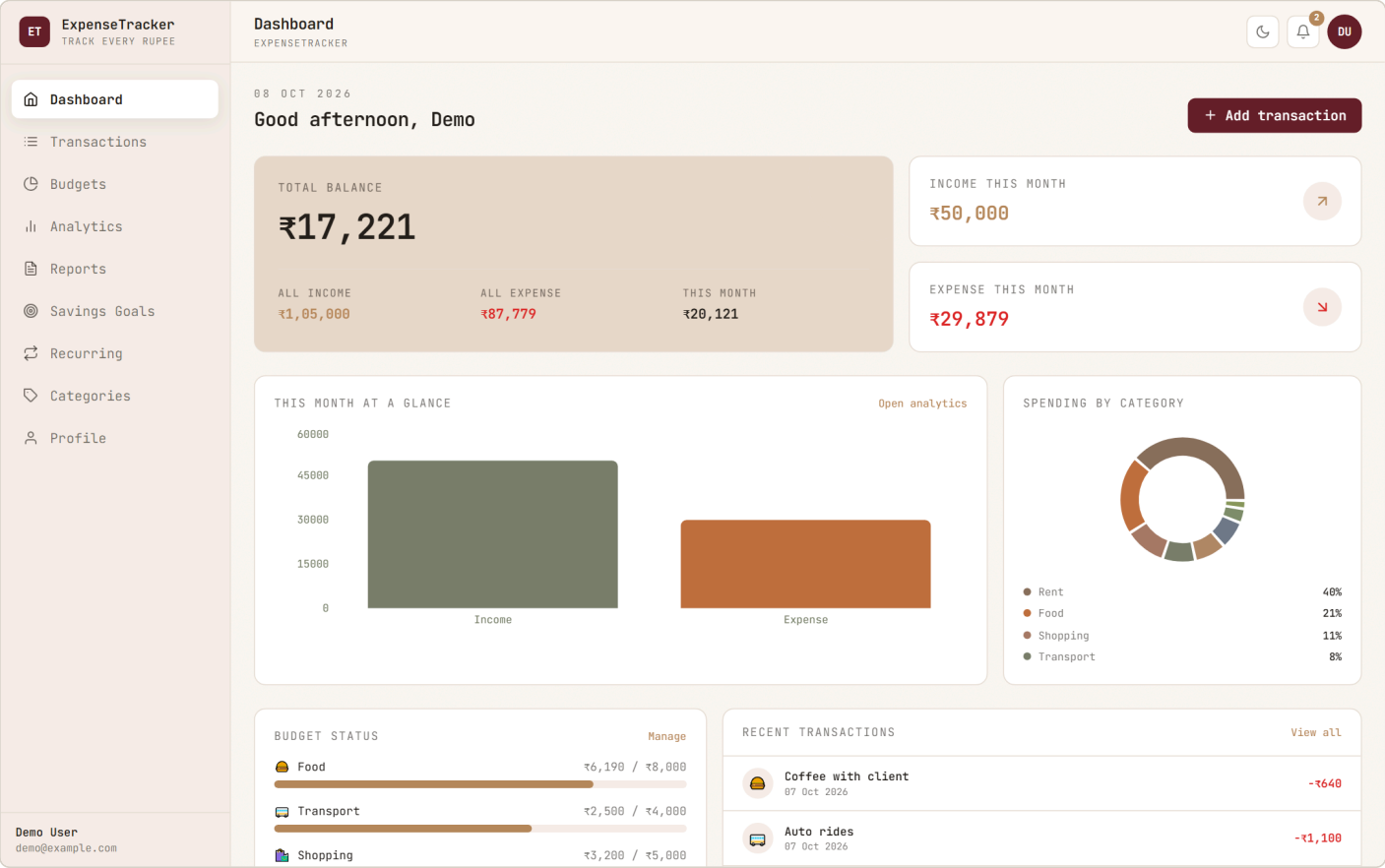


Figure 20 — An alternate theme (Cream Burgundy) demonstrating the pluggable theme engine

9.4.11 UX States

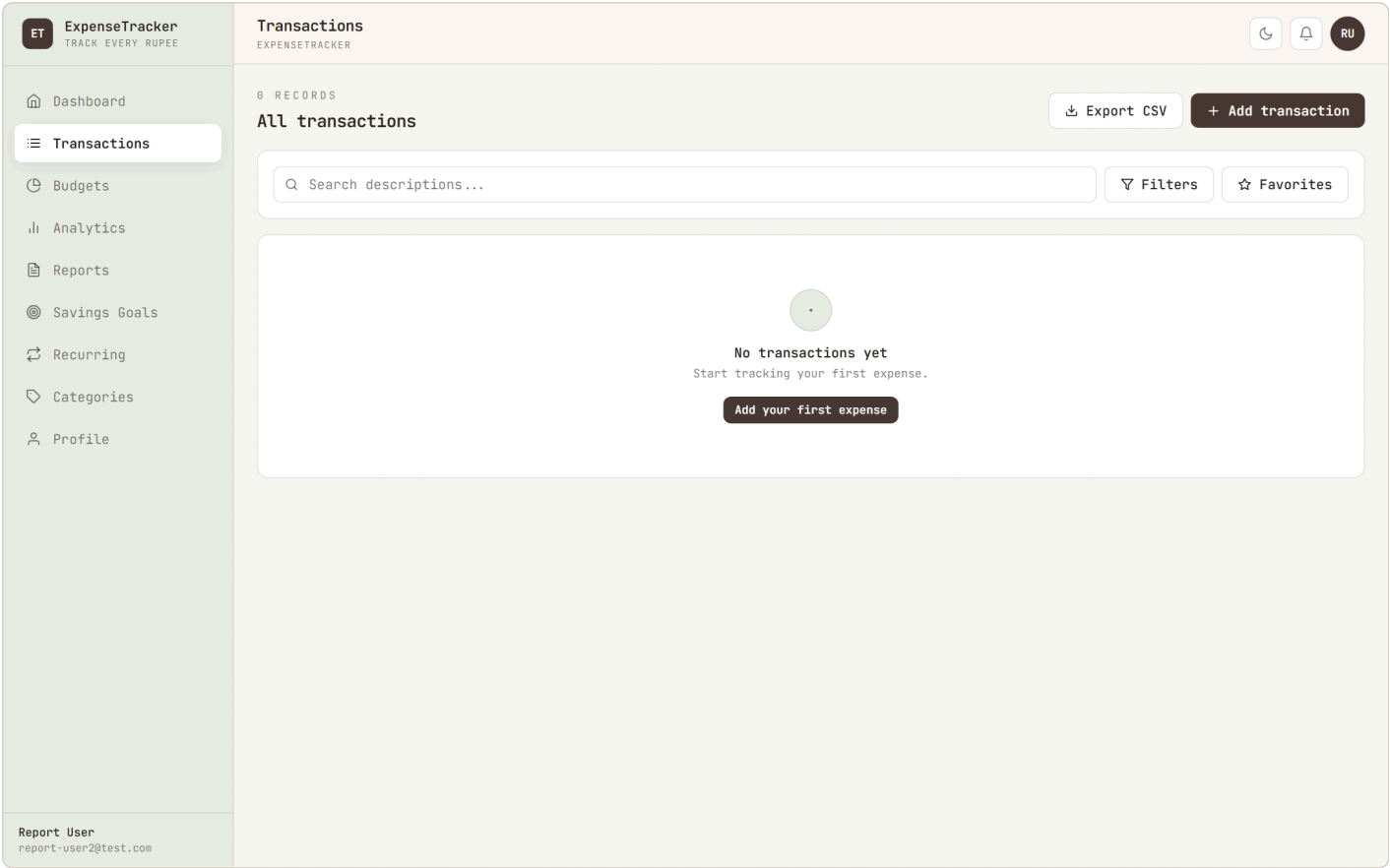


Figure 21 — Friendly empty state instead of a blank table (every list page has one, plus loading skeletons)

Category	Screens	Figures
Authentication	1	7
Dashboard	2 (light + dark)	8–9
Transactions	2 (list + modal)	10–11
Budgets	1	12
Analytics & Charts	2 (bar + line)	13–14
Monthly Reports	1	15
Savings Goals	1	16
Recurring Expenses	1	17
Categories	1	18
Appearance & Themes	2 (selector + theme)	19–20
UX States	1	21
Total	15 screens	Figures 7–21

10. Implementation Highlights (with code)

The seven annotated code figures shipped with this report are grouped by tier below — four from the backend (server logic) and three from the frontend (React / theming / validation).

10.1 Backend (Server-side) Implementation

10.1.1 Recurring Expense Auto-Posting

Whenever the dashboard is loaded, every active recurring item whose `nextDate` is today or earlier is converted into a real transaction, its next due date is advanced by one frequency step, and a notification is created. The posted transaction keeps a link back to its recurring parent so the UI can mark it.

```
const today = new Date();
today.setHours(0, 0, 0, 0);

const dueItems = await RecurringExpense.find({
  user: userId,
  active: true,
  nextDate: { $lte: today },
});

for (const item of dueItems) {
  await Transaction.create({
    user: userId,
    type: 'expense',
    amount: item.amount,
    category: item.category,
    description: item.title,
    date: item.nextDate,
    paymentMethod: item.paymentMethod,
    isRecurring: true,
    recurringExpense: item._id,
  });

  item.nextDate = nextDateAfter(item.nextDate, item.frequency);
  await item.save();

  await recurringPosted(userId, item.title, item.amount);
}
```

server/src/utils/processRecurring.js — Recurring expense auto-posting

Figure 22 — `server/src/utils/processRecurring.js` — auto-posting of due recurring expenses

```

const dueItems = await RecurringExpense.find({
  user: userId,
  active: true,
  nextDate: { $lte: today },
});

for (const item of dueItems) {
  await Transaction.create({
    user: userId, type: 'expense',
    amount: item.amount, category: item.category,
    description: item.title, date: item.nextDate,
    paymentMethod: item.paymentMethod,
    isRecurring: true, recurringExpense: item._id,
  });

  item.nextDate = nextDateAfter(item.nextDate, item.frequency);
  await item.save();
}

```

10.1.2 Smart Notifications with De-duplication

Each alert carries a stable key such as `budget:2026:10:Food:warn`. Before creating a notification the server checks whether that key already exists, so the same warning never repeats inside a month — the bell stays useful instead of noisy.

```

const Notification = require('../models/Notification');

const createOnce = async (userId, type, title, message, key) => {
  const alreadySent = await Notification.findOne({ user: userId, key });
  if (alreadySent) return;

  await Notification.create({ user: userId, type, title, message, key });
};

const checkBudgets = async (userId, month, year, budgets) => {
  for (const item of budgets) {
    const categoryName = item.category ? item.category.name : 'Budget';

    if (item.percentage >= 100) {
      await createOnce(
        userId,
        'budget_exceeded',
        'Budget exceeded',
        `${categoryName} budget is over by ${Math.round(item.percentage - 100)}%`,
        `budget:${year}:${month}:${categoryName}:over`
      );
    } else if (item.percentage >= 80) {
      await createOnce(
        userId,
        'budget_warning',
        'Budget warning',
        `${categoryName} budget is ${Math.round(item.percentage)}% used`,
        `budget:${year}:${month}:${categoryName}:warn`
      );
    }
  }
}

```

server/src/utils/notify.js — Smart notification helper with de-duplication

Figure 23 — `server/src/utls/notify.js` — create-once helper with stable de-duplication keys

```
const createOnce = async (userId, type, title, message, key) => {
  const alreadySent = await Notification.findOne({ user: userId, key });
  if (alreadySent) return;

  await Notification.create({ user: userId, type, title, message, key });
};
```

Key formats: `budget:{year}:{month}:{category}:warn` ($\geq 80\%$), `budget:{year}:{month}:{category}:over` ($\geq 100\%$), `goal:{id}:reached`, and `recurring:{userId}:{title}:{timestamp}`.

10.1.3 Dashboard & Report Aggregation

Totals, six-month series and category breakdowns are computed with MongoDB aggregation pipelines executed with sequential `await` calls for readability. Categories are joined with `$lookup` so name, icon and colour travel with each row.

```

    await checkBudgets(userId, month, year, budgetStatus);

    res.json({
      success: true,
      data: {
        balance: allTime.income - allTime.expense,
        totalIncome: allTime.income,
        totalExpense: allTime.expense,
        monthlyIncome: monthTotals.income,
        monthlyExpense: monthTotals.expense,
        monthlySavings: monthTotals.income - monthTotals.expense,
        categoryBreakdown,
        recentTransactions,
        budgetStatus,
      },
    });
  } catch (error) {
    next(error);
  }
};

exports.getMonthly = async (req, res, next) => {
  try {
    const year = parseInt(req.query.year) || new Date().getFullYear();
    const startDate = new Date(year, 0, 1);
    const endDate = new Date(year, 11, 31, 23, 59, 59, 999);

    const rows = await Transaction.aggregate([
      { $match: { user: req.user._id, date: { $gte: startDate, $lte: endDate } } },
      { $group: { _id: { month: { $month: '$date' }, type: '$type' }, total: { $sum: '$amount' } } },
    ]);

    const result = [];
    for (let m = 1; m <= 12; m++) {
      const monthRow = { month: m, name: monthNames[m - 1], income: 0, expense: 0, savings: 0 };
      for (const row of rows) {
        if (row._id.month !== m) continue;
        if (row._id.type === 'income') monthRow.income = row.total;
        if (row._id.type === 'expense') monthRow.expense = row.total;
      }
      monthRow.savings = monthRow.income - monthRow.expense;
      result.push(monthRow);
    }

    res.json({ success: true, data: result });
  } catch (error) {
    next(error);
  }
};

exports.getCategoriesAnalytics = async (req, res, next) => {
  try {
    const now = new Date();
    const month = parseInt(req.query.month) || now.getMonth() + 1;
    const year = parseInt(req.query.year) || now.getFullYear();

```

server/src/controllers/analyticsController.js — Dashboard aggregation (sequential awaits, no Promise.all)

Figure 24 — `server/src/controllers/analyticsController.js` — dashboard aggregation

```
const categoryRows = await Transaction.aggregate([
  { $match: { user: userId, type: 'expense',
    date: { $gte: win.start, $lte: win.end } } },
  { $lookup: { from: 'categories', localField: 'category',
    foreignField: '_id', as: 'categoryDoc' } },
  { $unwind: '$categoryDoc' },
  { $group: {
    _id: '$category',
    name: { $first: '$categoryDoc.name' },
    icon: { $first: '$categoryDoc.icon' },
    color: { $first: '$categoryDoc.color' },
    total: { $sum: '$amount' } } },
  { $sort: { total: -1 } },
]);
```

Bug fixed here: inside `$group`, non-`_id` fields must be accumulated — a bare path raises *"The field 'x' must be an accumulator object"*. Fields such as `name` and `type` therefore use `{ $first: '$field' }`. The same correction was applied in three pipelines (dashboard, monthly series and reports).

10.1.4 CSV Export

```

    if (!transaction) {
      return res.status(404).json({ success: false, message: 'Transaction not found' });
    }

    if (req.body.amount !== undefined) {
      const validAmount = toAmount(req.body.amount);
      if (validAmount === null) {
        return res.status(400).json({ success: false, message: 'Please provide a valid amount greater than 0' });
      }
      transaction.amount = validAmount;
    }

    const allowedFields = ['type', 'category', 'description', 'date', 'paymentMethod', 'tags', 'notes', 'receipt', 'isFavorite'];
    for (const field of allowedFields) {
      if (req.body[field] !== undefined) {
        transaction[field] = req.body[field];
      }
    }

    await transaction.save();
    await transaction.populate('category', 'name icon color type');
    await alertBudgetsFor(req.user, new Date(transaction.date));

    res.json({ success: true, data: transaction });
  } catch (error) {
    next(error);
  }
};

exports.deleteTransaction = async (req, res, next) => {
  try {
    const transaction = await Transaction.findOneAndDelete({ _id: req.params.id, user: req.user.id });

    if (!transaction) {
      return res.status(404).json({ success: false, message: 'Transaction not found' });
    }

    removeReceiptFile(transaction.receipt);
  } catch (error) {
    next(error);
  }
};

```

server/src/controllers/transactionController.js — CSV export endpoint

Figure 25 — `server/src/controllers/transactionController.js` — CSV export endpoint

```

const buildCsv = (transactions) => {
  const header = ['Date', 'Type', 'Description', 'Category',
    'Amount', 'Payment Method', 'Tags', 'Notes'];
  const rows = [header.join(',')];

  for (const item of transactions) {
    const values = [ /* ...8 columns... */ ];
    const line = values
      .map((v) => `"${String(v).replace(/"/g, '"')}")`)
      .join(',');
    rows.push(line);
  }
  return rows.join('\n');
};

```

10.2 Frontend (Client-side) Implementation

10.2.1 Amount Validation on Both Ends

```
export const sanitizeAmountInput = (value) => {
  let cleaned = String(value).replace(/^[^0-9.]/g, '');

  const parts = cleaned.split('.');
  if (parts.length > 2) {
    cleaned = parts[0] + '.' + parts.slice(1).join('');
  }

  const [whole, decimal] = cleaned.split('.');
  if (decimal !== undefined) {
    cleaned = whole + '.' + decimal.slice(0, 2);
  }

  return cleaned;
};

export const isValidAmount = (value) => {
  const number = Number(value);
  return value !== '' && Number.isFinite(number) && number > 0;
};

export const amountErrorMessage = (value) => {
  if (value === '' || value === null || value === undefined) return 'Amount is required';
  const number = Number(value);
  if (!Number.isFinite(number)) return 'Please enter a valid number';
  if (number <= 0) return 'Amount must be greater than 0';
  return '';
};
```

client/src/utils/amount.js — Amount sanitization helper

Figure 26 — `client/src/utils/amount.js` — client-side amount sanitization

```
// server/src/utils/sanitize.js
const toAmount = (value) => {
  const number = Number(value);
  if (!Number.isFinite(number) || number <= 0) return null;
  return Math.round(number * 100) / 100; // 123.456 -> 123.46
};
```

The client's `sanitizeAmountInput` strips letters and minus signs while typing and caps the decimal places at two; the server independently converts and rejects invalid values with HTTP 400 — so `abc`, `-5` or a zero budget never reach the database.

10.2.2 Theme File = One File per Theme

```

const theme = {
  id: 'option8',
  name: 'Option 8 — Warm White + Espresso + Sage',
  light: {
    bg: '#FAF8F3',
    text: '#29231F',
    card: 'FFFFFF',
    border: '#29231F1F',
    primary: '#493B32',
    primaryText: 'FFFFFF',
    primaryHover: '#3A2F27',
    accent: '#78816A',
    sage: '#E9EDE4',
    beige: '#E9DFD1',
  },
  dark: {
    bg: '#1E1A16',
    text: '#EFE9E1',
    card: '#28221C',
    border: '#EFE9E11F',
    primary: '#7A6656',
    primaryText: 'FFFFFF',
    primaryHover: '#8C7767',
    accent: '#9BA586',
    sage: '#333D2F',
    beige: '#3A322A',
  },
};

export default theme;

```

client/src/themes/option8WarmEspressoSage.js — One theme = one separate file

Figure 27 — `client/src/themes/option8WarmEspressoSage.js` — one theme = one separate file

```

export const applyTheme = (theme, dark) => {
  const palette = dark ? theme.dark : theme.light;
  const root = document.documentElement;

  root.style.setProperty('--bg', palette.bg);
  root.style.setProperty('--text', palette.text);
  root.style.setProperty('--card', palette.card);
  root.style.setProperty('--primary', palette.primary);
  root.style.setProperty('--accent', palette.accent);
  // ... 12 variables in total
};

```

10.2.3 Instant Chart Selector

```

</div>

<div className="grid gap-4 lg:grid-cols-2">
  <div className="card p-5">
    <div className="mb-4 flex items-center gap-2">
      <FiPieChart size={14} className="text-accent" />
      <p className="section-title">Spending breakdown · {monthName(now.getMonth() + 1)}</p>
    </div>
    {breakdown.length === 0 ? (
      <EmptyState title="No spending this month" message="Expenses will appear here once you add them." />
    ) : (
      <div className="space-y-3">
        {breakdown.map((item) => (
          <div key={item._id}>
            <div className="mb-1 flex items-center justify-between text-xs">
              <span className="flex items-center gap-2">
                <span>{item.icon}</span> {item.name}
              </span>
              <span className="font-medium">
                {formatCurrency(item.total, currency)} · {Math.round(item.percentage)}%
              </span>
            </div>
            <div className="h-1.5 w-full overflow-hidden rounded-full bg-sage">
              <div
                className="h-full rounded-full transition-all duration-300"
                style={{ width: `${item.percentage}%`, backgroundColor: item.color }}
              />
            </div>
          </div>
        ))}
      </div>
    )}
  </div>

  <div className="card p-5">
    <div className="mb-4 flex items-center gap-2">

```

client/src/pages/Analytics.jsx — Vertical multi-chart selector

Figure 28 — `client/src/pages/Analytics.jsx` — vertical multi-chart selector

Analytics keeps the fetched data mounted and only swaps the chart component (Bar / Line / Area / Donut / Pie), so switching views is instant and never re-fetches from the server.

10.2.4 Axios Interceptors (single HTTP client)

```

API.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

API.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('token');
      if (window.location.pathname !== '/login')
        window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

```

There is exactly one HTTP client in the application, so authentication behaviour is defined in one place instead of being repeated in every thunk.

10.3 Code Figure Index (category-wise)

Figure	Category	Source file	Shows
22	Backend (server)	server/src/utils/processRecurring.js	Auto-posting of due recurring expenses
23		server/src/utils/notify.js	Create-once notifications with de-dup keys
24		server/src/controllers/analyticsController.js	Dashboard aggregation pipeline
25		server/src/controllers/transactionController.js	CSV export endpoint
26	Frontend (client)	client/src/utils/amount.js	Amount sanitization while typing
27		client/src/themes/option8WarmEspressoSage.js	One theme = one file (light + dark palettes)
28		client/src/pages/Analytics.jsx	Vertical multi-chart selector
—		client/src/services/api.js	Axios request/response interceptors (printed above)

11. Security & Data Validation

Control	Implementation
Password storage	bcryptjs with 10 salt rounds inside a <code>pre('save')</code> hook; the field is <code>select:false</code> so it is never returned by any API response.
Session / auth	HS256 JWT signed with <code>JWT_SECRET</code> , 7-day expiry, sent as <code>Authorization: Bearer</code> ; verified on every protected request along with a user-existence check. All 401 responses are deliberately generic (no signature-vs-expiry distinction).
Ownership scoping	Every find / update / delete includes <code>user: req.user._id</code> — a user can never read or modify another user's records even with a valid token.
Input validation	Mongoose schema validators (required, min/max, maxlength, enum, e-mail regex) plus <code>toAmount()</code> on numeric fields; invalid values return HTTP 400 with clear messages.
Mass-assignment protection	Controllers copy only whitelisted fields (e.g. <code>['type','category','description','date','paymentMethod','tags','notes','receipt','isFavorite']</code>) instead of spreading the request body.
HTTP hardening	helmet v7 default headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy ...) with only <code>Cross-Origin-Resource-Policy</code> relaxed for local images; CORS allow-list restricted to the single client URL; morgan request logging in development.
File uploads	Multer disk storage with an image-only MIME filter, a 5 MB size limit and a server-generated filename (<code>timestamp-random.ext</code>), so the user never controls the path; deletion uses <code>path.basename()</code> before <code>fs.unlink</code> to prevent path traversal.
Error handling	A central error handler maps <code>CastError</code> → 404, duplicate key (11000) → 400, <code>ValidationError</code> → 400 and never leaks stack traces to the client.
Configuration	Environment variables are read in exactly two files (<code>server/src/config/env.js</code> , <code>client/src/config/env.js</code>); secrets stay in a git-ignored <code>.env</code> with a committed <code>.env.example</code> .

TC	Area	Test scenario	Expected result	Status
TC01	Health	GET /api/health	200 OK with status payload	Pass
TC02	Auth	Register a new user with valid details	200, JWT returned, user created	Pass
TC03	Auth	Login with wrong password	401 with generic message	Pass
TC04	Auth	Call a protected route without a token	401 "Not authorized, no token"	Pass
TC05	CRUD	Create / update / delete transaction, budget, goal, recurring plan, category	200 at each step; list reflects the change	Pass
TC06	Dashboard	GET /analytics/dashboard after the accumulator fix	200 with totals, series, category breakdown, budget status	Pass
TC07	Reports	GET /analytics/reports for a month with a previous month present	200 with month-over-month % and category change %	Pass
TC08	Validation	Amount = abc or -5	Rejected with HTTP 400	Pass
TC09	Validation	Amount = 123.456	Stored as 123.46 (2 dp rounding)	Pass
TC10	Validation	Budget amount = 0	Rejected (min 0 for update, positive for create)	Pass
TC11	Isolation	Fetch records of user A using user B's token	Empty / not found — no cross-user data	Pass
TC12	Recurring	Load dashboard with a past-due recurring plan	Transaction auto-posted, next date advanced, notification created	Pass
TC13	Notifications	Refresh dashboard repeatedly with a budget ≥80%	Only one warning per month (de-duplicated)	Pass
TC14	Categories	Delete a category used by a recurring expense	Blocked with a clear message	Pass
TC15	CSV	GET /api/transactions/csv with active filters	Valid CSV stream, correctly escaped quotes	Pass
TC16	Upload	Upload a non-image file / file > 5 MB	Rejected by the Multer file filter	Pass

TC17	Frontend	<code>npm run build</code> (production bundle)	Build success (948 modules)	Pass
TC18	Seeding	<code>npm run seed</code> then log in with both demo accounts	Demo: 32 transactions / Saurabh: 20 transactions verified	Pass
TC19	Visual	Every page captured in light and dark mode	15 screenshots rendered correctly	Pass
TC20	Accessibility	Keyboard navigation: Tab through forms, Escape to close modals	Focus ring visible; modals close on Escape	Pass

13. Challenges Faced & Solutions

Challenge	Solution adopted
MongoDB error <i>"The field must be an accumulator object"</i> inside <code>\$group</code> when adding plain fields such as <code>name</code> .	Used accumulators such as <code>{ \$first: '\$name' }</code> for every non- <code>_id</code> field in three aggregation pipelines; verified with <code>GET /analytics/dashboard</code> .
Custom CSS class <code>.table-row</code> clashed with Tailwind's generated <code>display: table-row</code> utility and broke a flex row.	Renamed the component class to <code>.row-hover</code> ; the built CSS now contains zero unintended <code>table-row</code> rules.
Amount fields accepted letters, negatives and unlimited decimals, corrupting totals.	Added <code>sanitizeAmountInput</code> on the client and <code>toAmount</code> on the server, then tested with API calls (TC08–TC10).
The same budget warning reappeared on every refresh, flooding the bell dropdown.	Stable notification keys with a pre-insert existence check (de-duplication) in <code>notify.js</code> .
Port conflicts (<code>EADDRINUSE</code>) when restarting the development servers.	Exactly one dev process per port, ports defined in <code>.env</code> , and the Vite proxy hides the backend port from the browser.
Environment variables were scattered across many files, making configuration hard to audit.	Restricted raw <code>process.env</code> / <code>import.meta.env</code> access to exactly two config files with defaults.
Deleting a category that was still referenced produced broken transactions.	Server-side check blocks the delete while a recurring expense uses the category; categories are referenced, not copied.
Receipt deletion could escape the uploads directory if the stored name was manipulated.	Always pass the file name through <code>path.basename()</code> before <code>fs.unlink</code> .

14. Limitations & Future Scope

14.1 Limitations of the Present System

- Recurring expenses are posted only when the app is opened — there is no background scheduler.
- The production build shows a chunk-size warning (>500 kB) because Recharts bundles all chart types in one chunk.
- Only monthly budgets, a single currency per user and no live exchange-rate conversion.
- No automated test suite yet; verification was done manually and with API calls (Chapter 12).
- Receipt images are stored on local disk rather than in the database or cloud object storage.
- No rate limiting on the authentication endpoints (planned with `express-rate-limit`).
- The application is not yet deployed to a public server; it runs on localhost.

14.2 Future Scope

1. Cron scheduler (`node-cron`) to post recurring expenses without opening the app.
2. Monthly report e-mails and budget-breach push / e-mail alerts.
3. Bank statement (CSV/OFX) import and receipt OCR for automatic entry.
4. Smart features: automatic category suggestion, anomaly detection and month-end forecasting.
5. Multi-currency support with daily exchange rates and shared / family budgets.
6. PWA with offline mode and a React Native mobile app using the same REST API.
7. Rate limiting, refresh-token rotation and an audit log of edits/deletes.
8. Jest + Supertest API tests, React Testing Library component tests and a CI/CD pipeline.
9. Route-level code splitting to remove the bundle-size warning.
10. Gamification (streaks, savings badges) and chart image export for presentations.

15. Conclusion

ExpenseTracker delivers an end-to-end MERN application covering the complete personal-finance workflow — transactions, categories, budgets, savings goals, recurring payments, analytics, monthly reports and notifications — with secure per-user data, validation on both client and server, and a themeable, accessible interface built on a distinctive design system.

The project demonstrated the practical application of the complete software-development lifecycle: requirement analysis, feasibility study, use-case and ER modelling, three-tier design, iterative implementation, systematic testing (20 test cases) and documentation. Technical problems encountered during development — MongoDB aggregation accumulator rules, CSS class collisions, amount validation and notification de-duplication — were diagnosed and resolved, and each resolution is recorded in Chapter 13.

The modular structure (small single-purpose files, sequential `async/await` flow, centralised configuration and a single HTTP client) keeps the codebase easy to understand, maintain and extend. The working system, verified screenshots and test-case results confirm that the stated objectives of Chapter 2 have been achieved, while Chapter 14 provides a clear roadmap for growth beyond the current semester's requirements.

References

1. React documentation — *React – A JavaScript library for building user interfaces*, <https://react.dev>
2. Redux Toolkit documentation — *Usage Guide*, <https://redux-toolkit.js.org>
3. Express.js documentation — <https://expressjs.com/en/guide/routing.html>
4. Mongoose documentation — *Queries, Aggregation and Indexes*, <https://mongoosejs.com>
5. MongoDB Manual — *Aggregation Pipeline*, <https://www.mongodb.com/docs/manual/aggregation/>
6. JSON Web Token (RFC 7519) — <https://datatracker.ietf.org/doc/html/rfc7519>
7. bcryptjs package documentation — <https://www.npmjs.com/package/bcryptjs>
8. Multer documentation — *Handling file uploads*, <https://github.com/expressjs/multer>
9. Tailwind CSS documentation — *Styling with CSS variables & dark mode*, <https://tailwindcss.com/docs>
10. Recharts documentation — <https://recharts.org/en-US>
11. Vite documentation — <https://vitejs.dev/guide>
12. MDN Web Docs — *HTTP authentication, CORS and Content Security Policy*, <https://developer.mozilla.org>
13. OWASP Cheat Sheet Series — *Authentication Cheat Sheet*, <https://cheatsheetseries.owasp.org>
14. Gujarat Technological University — Mini Project guidelines, Semester VII (2025–26).

Appendix A — How to Run & Demo Credentials

Prerequisites: Node.js $\geq$ 18 and MongoDB running locally on port 27017.

```
# 1. Backend
cd server
npm install
cp .env.example .env
npm run seed      # creates 2 demo users with realistic data
npm run dev       # http://localhost:5000

# 2. Frontend (new terminal)
cd client
npm install
npm run dev       # http://localhost:5173
```

Demo account	E-mail	Password	Seeded data
Demo User	demo@example.com	password123	32 transactions, 5 budgets, 5 recurring plans, 3 goals, 19 categories
Saurabh	saurabh@gmail.com	saurabh123	20 transactions, 3 budgets, 3 recurring plans, 3 goals, 19 categories

Production build: `cd client && npm run build` (output in `client/dist`). Report PDF is generated from this HTML with the repository's Playwright helper.

Appendix B — Environment Variables & Seed Data

server/.env

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/expenseTracker
JWT_SECRET=<your-secret>
JWT_EXPIRE=7d
CLIENT_URL=http://localhost:5173
UPLOAD_DIR=uploads
```

client/.env

```
VITE_API_BASE_URL=/api
VITE_APP_NAME=ExpenseTracker
VITE_APP_TAGLINE=Track every rupee
VITE_REQUEST_TIMEOUT=15000
```

Seed Data Summary

Item	Contents
Users	2 (Demo User — monthly income ₹50,000; Saurabh — ₹75,000), both currency INR
Categories	19 per user — 13 expense (Food, Transport, Shopping, Rent, Bills, Entertainment, Health, Education, Travel, Subscriptions, Clothing, Mobile, Other) + 6 income (Salary, Freelance, Investment, Gift, Interest, Other Income)
Transactions	52 total across September & October 2026 (Demo 32, Saurabh 20) so charts, reports and month-over-month comparisons all have content
Budgets	8 (Demo 5, Saurabh 3) for October 2026 — e.g. Food ₹8,000, Transport ₹4,000, Shopping ₹5,000
Savings goals	6 (3 each) — e.g. New Laptop ₹80,000/₹45,000, Goa Trip ₹30,000/₹18,000, Emergency Fund
Recurring plans	8 (Demo 5, Saurabh 3), all monthly and active; two are already past-due so auto-posting demonstrates itself on first login
Notifications	0 seeded — created at runtime by the budget / goal / recurring rules

Appendix C — Code Metrics & Module Index

Metric	Value
REST endpoints	41 (40 in routers + health check); 38 authenticated
Route files / controllers / models / utils / middleware	8 / 8 / 7 / 6 / 3
Frontend pages / Redux slices / themes / shared components	11 / 8 / 8 / 11
Client routes	12 (2 public, 9 protected under <code>MainLayout</code> , 1 wildcard redirect)
Largest source files	<code>Transactions.jsx</code> (~20 KB), <code>Reports.jsx</code> (~14 KB), <code>analyticsController.js</code> (336 lines)
Production build	948 modules bundled successfully (Vite 5)
Screenshots captured for this report	15 UI + 7 annotated code figures

Backend module index

Path	Responsibility
<code>server/src/config/env.js</code>	The only file allowed to read <code>process.env</code> ; supplies defaults for 7 variables.
<code>server/src/config/db.js</code>	<code>mongoose.connect</code> with failure exit code 1.
<code>server/src/middleware/auth.js</code>	<code>protect</code> — JWT verification and <code>req.user</code> hydration.
<code>server/src/middleware/error.js</code>	Central error handler (CastError / 11000 / ValidationError mapping).
<code>server/src/middleware/upload.js</code>	Multer disk storage, image MIME filter, 5 MB limit.
<code>server/src/utils/processRecurring.js</code>	Auto-posts due recurring expenses and advances <code>nextDate</code> .
<code>server/src/utils/notify.js</code>	Create-once notification helpers with stable keys.
<code>server/src/utils/budgetStatus.js</code>	Computes spent / percentage / remaining per budget.
<code>server/src/utils/buildCsv.js</code>	CSV serialiser with RFC-style quote escaping.
<code>server/src/utils/sanitize.js</code>	<code>toAmount</code> — finite, positive, two-decimal rounding.
<code>server/src/utils/defaultCategories.js</code>	19 default categories used by the seeder.
<code>server/src/seed/seed.js</code>	Wipes the database and recreates both demo accounts with data.

Frontend module index

Path	Responsibility
<code>client/src/App.jsx</code>	Route table, <code>ProtectedRoute</code> / <code>PublicRoute</code> guards, theme application and <code>getMe()</code> hydration.
<code>client/src/services/api.js</code>	The single Axios instance with token and 401 interceptors.
<code>client/src/store/index.js</code>	<code>configureStore</code> combining eight slices.
<code>client/src/themes/index.js</code>	Theme registry: <code>themes</code> , <code>getTheme()</code> , <code>applyTheme()</code> .
<code>client/src/index.css</code>	CSS variables, Tailwind layers, component classes, keyframes, reduced-motion rule.
<code>client/src/hooks/useDebounce.js</code>	300 ms debounce used by the transaction search box.
<code>client/src/utils/amount.js</code>	<code>sanitizeAmountInput</code> , <code>isValidAmount</code> , <code>amountErrorMessage</code> .
<code>client/src/components/layout/*</code>	Sidebar (9 nav links), Navbar (theme toggle, notifications, profile), <code>MainLayout</code> .

— End of Report —

ExpenseTracker · Mini Project Report · Sharma Saurabh (250863131007) · GTU 2025–26